

Modélisation du risque de crédit — Estimation conjointe PD · LGD · EAD & calcul ECL IFRS 9

ABDOULAYE TANGARA

Fintech Product Manager &

Student in MSc in Quantitative and Computable Economics

Contacts

 [LinkedIn](#)

 [Mon GitHub](#)

 [Mon Portfolio](#)

 abdoulayetangara722@gmail.com



*"To get something you never had, you've got to do something you never
did. Get your mind right ! "*

Introduction

Ce document présente une modélisation complète du risque de crédit appliquée aux données LendingClub (2007–2019), couvrant l’estimation conjointe des trois paramètres fondamentaux exigés par les cadres réglementaires Bâle II/III et IFRS 9 : la probabilité de défaut (PD), la perte en cas de défaut (LGD) et l’exposition au moment du défaut (EAD). À partir d’un portefeuille de 2,26 millions de prêts, le travail s’articule en trois étapes successives — ingestion et feature engineering, analyse exploratoire statistique, puis modélisation prédictive — pour aboutir au calcul de la perte attendue (ECL) et au staging IFRS 9 de l’ensemble du portefeuille. L’ensemble du pipeline a été développé sous contrainte de ressources computationnelles limitées, ce qui a imposé des choix d’optimisation rigoureux à chaque étape et renforce la reproductibilité du code produit.

1 Data Engineering & Feature Store

Objectif

Cette première section constitue le **premier maillon de la chaîne de modélisation du risque de crédit**. Il prend en charge :

1. **Ingestion & audit qualité** — chargement, typage, inventaire des données manquantes ;
2. **Détection & traitement des outliers** — méthode IQR adaptative sur toutes les variables financières;
3. **Imputation supervisée** — stratégie différenciée selon la distribution et le taux de missing;
4. **Création de la variable cible PD** — binarisation conforme Bâle II (90 DPD);
5. **Feature engineering** — interactions, ratios financiers, indicateurs de stress;
6. **Exclusion des variables contaminantes** — data leakage, informations post-défaut, données sensibles.

Convention de nommage : toutes les colonnes créées dans ce notebook sont préfixées `fe_` (feature-engineered) pour traçabilité.

1.1 Environnement & Configuration

```
[ ]: # =====
# 0.1 IMPORTS
# =====

import os
import warnings
import logging
from pathlib import Path
from datetime import datetime
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
from google.colab import drive
warnings.filterwarnings('ignore')
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)s | %(message)s',
    datefmt='%H:%M:%S'
)
log = logging.getLogger(__name__)
np.random.seed(42)

# =====
# 0.2 STYLE GRAPHIQUE
# =====

MS_BLUE    = '#003087'
MS_LIGHT   = '#6699CC'
MS_GREY    = '#666666'
MS_RED     = '#CC0000'
MS_GREEN   = '#006633'
MS_AMBER   = '#CC8800'

plt.rcParams.update({
    'figure.facecolor'      : 'white',
    'axes.facecolor'        : 'white',
    'axes.edgecolor'        : '#CCCCCC',
    'axes.linewidth'        : 0.8,
    'axes.spines.top'        : False,
    'axes.spines.right'     : False,
    'axes.grid'              : True,
    'grid.color'             : '#EEEEEE',
    'grid.linewidth'         : 0.5,
    'font.family'            : 'sans-serif',
```

```

        'font.size'           : 11,
        'axes.titlesize'      : 13,
        'axes.titleweight'    : 'bold',
        'axes.labelsize'      : 11,
        'xtick.labelsize'     : 10,
        'ytick.labelsize'     : 10,
        'legend.frameon'      : False,
    })

# =====
# 0.3 CONFIGURATION DES CHEMINS
# =====

drive.mount('/content/drive', force_remount=True)

BASE_PATH    = Path('/content/drive/MyDrive/Credit_Risk_PD_LGD_EAD')
RAW_PATH     = BASE_PATH / 'DATA' / 'RAW'
INTERIM_PATH = BASE_PATH / 'DATA' / 'INTERIM'
REPORTS_PATH = BASE_PATH / 'REPORTS'

for p in [INTERIM_PATH, REPORTS_PATH]:
    p.mkdir(parents=True, exist_ok=True)

log.info('Environnement initialisé | Base: %s', BASE_PATH)

```

Mounted at /content/drive

1.2 Ingestion & Audit Qualité

```

[ ]: # =====
# 1.1 CHARGEMENT OPTIMISÉ MÉMOIRE
# =====

log.info('Chargement du dataset brut...')
df_raw = pd.read_csv(RAW_PATH / 'loan.csv', low_memory=False)

# Typage initial - réduction empreinte mémoire
CAT_COLS = [
    'grade', 'sub_grade', 'emp_title', 'home_ownership', 'purpose', 'term',
    'verification_status', 'loan_status', 'pymnt_plan', 'initial_list_status',
    'application_type', 'hardship_flag', 'addr_state', 'disbursement_method',
    'debt_settlement_flag'
]
DATE_COLS = ['issue_d', 'earliest_cr_line', 'last_pymnt_d', 'last_credit_pull_d']

for col in CAT_COLS:
    if col in df_raw.columns:
        df_raw[col] = df_raw[col].astype('category')

```

```

for col in DATE_COLS:
    if col in df_raw.columns:
        df_raw[col] = pd.to_datetime(df_raw[col], format='%b-%Y',
→errors='coerce')

# emp_length - conversion numérique propre
df_raw['emp_length'] = (
    df_raw['emp_length']
    .str.replace(r'[^0-9]', '', regex=True)
    .replace('', np.nan)
    .astype(float)
    .fillna(0)
)

# Float64 → Float32 (gain ~50% mémoire numérique)
num_cols = df_raw.select_dtypes(include=['float64', 'int64']).columns
df_raw[num_cols] = df_raw[num_cols].astype(np.float32)

log.info('Dataset chargé : %s lignes × %s colonnes', *df_raw.shape)

```

```

[ ]: # =====
# 1.2 RAPPORT DE QUALITÉ DES DONNÉES
# =====
def data_quality_report(df: pd.DataFrame) -> pd.DataFrame:
    """
    Génère un rapport de qualité complet pour chaque colonne.
    Retourne un DataFrame trié par taux de missing décroissant.
    """
    n = len(df)
    report = pd.DataFrame({
        'dtype'          : df.dtypes,
        'n_missing'      : df.isnull().sum(),
        'pct_missing'    : df.isnull().mean() * 100,
        'n_unique'       : df.nunique(),
        'pct_unique'     : df.nunique() / n * 100,
    })
    # Ajouter skewness pour les numériques
    skew = df.select_dtypes(include='number').skew()
    report['skewness'] = skew

    return report.sort_values('pct_missing', ascending=False)

dq = data_quality_report(df_raw)

# Affichage - colonnes avec missing seulement
dq_missing = dq[dq['n_missing'] > 0].copy()
print(f'Colonnes avec valeurs manquantes : {len(dq_missing)} / {len(dq)}')

```

```
print(f'Taux global de complétude      : {(1 - df_raw.isnull().sum().sum() /
↳df_raw.size):.2%}')
display(dq_missing.head(30))
```

Colonnes avec valeurs manquantes : 112 / 145

Taux global de complétude : 66.89%

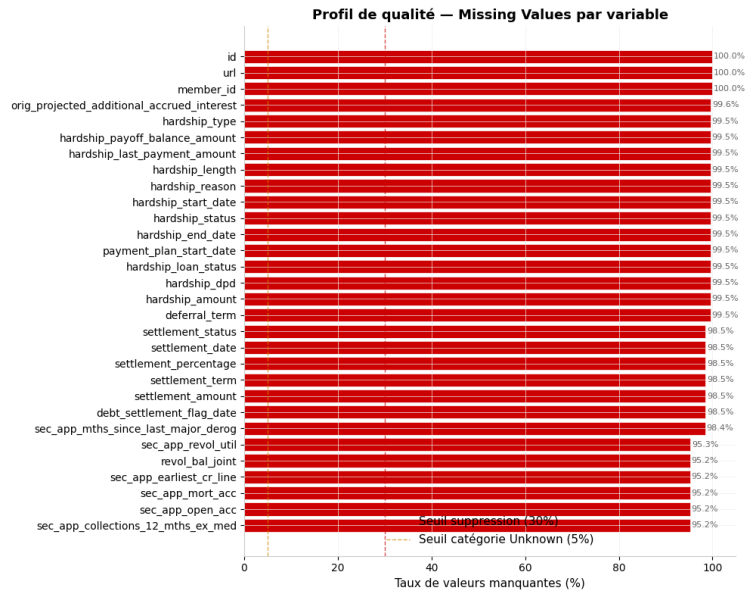
```
[ ]: # =====
# 1.3 VISUALISATION DU PROFIL DE MISSING VALUES
# =====
missing_top = dq_missing[dq_missing['pct_missing'] > 0].
↳sort_values('pct_missing', ascending=True)
missing_top = missing_top.tail(30) # Top 30

fig, ax = plt.subplots(figsize=(10, 8))
bars = ax.barh(missing_top.index, missing_top['pct_missing'],
               color=[MS_RED if v > 30 else MS_AMBER if v > 5 else MS_LIGHT
                       for v in missing_top['pct_missing']],
               edgecolor='white', linewidth=0.5)

ax.axvline(30, color=MS_RED, linestyle='--', linewidth=1, label='Seuil_
↳suppression (30%)', alpha=0.7)
ax.axvline(5, color=MS_AMBER, linestyle='--', linewidth=1, label='Seuil_
↳catégorie Unknown (5%)', alpha=0.7)

for bar, val in zip(bars, missing_top['pct_missing']):
    ax.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height()/2,
            f'{val:.1f}%', va='center', ha='left', fontsize=8, color=MS_GREY)

ax.set_xlabel('Taux de valeurs manquantes (%)')
ax.set_title('Profil de qualité - Missing Values par variable')
ax.legend()
plt.tight_layout()
plt.savefig(REPORTS_PATH / 'missing_profile.png', dpi=150, bbox_inches='tight')
plt.show()
```



1.3 Exclusion des Variables Contaminantes

```
[ ]: # =====
# 2.1 TAXONOMIE DES VARIABLES À EXCLURE

# =====
EXCLUDE = {
    'Identifiants_techniques': [
        'id', 'member_id', 'url', 'desc'],
    'Dates_futures': [
        'next_pymnt_d', 'hardship_start_date', 'hardship_end_date',
        'debt_settlement_flag_date', 'settlement_date', 'sec_app_earliest_cr_line'
    ],
    'Informations_sensibles': [
        'emp_title', # RGPD / égalité de traitement
    ],
    'Post_default_leakage': [
        # Statut de recouvrement (connu uniquement APRÈS défaut)
        'settlement_status', 'settlement_amount', 'settlement_percentage',
        ↪ 'settlement_term',
        'hardship_type', 'hardship_reason', 'hardship_status', 'deferral_term',
        'hardship_amount', 'hardship_length', 'hardship_dpd',
        ↪ 'hardship_loan_status',
        'orig_projected_additional_accrued_interest',
        ↪ 'hardship_payoff_balance_amount',
        'hardship_last_payment_amount',
        # Variables de comportement post-origination (SICR / observation window)
        'mths_since_last_major_derog', 'mths_since_recent_bc_dlq',
        'mths_since_recent_revolut_delinq',
```

```

        # Données de co-emprunteur (disponibles seulement pour joint_
        ↪applications)
        'annual_inc_joint', 'dti_joint', 'verification_status_joint',
        ↪'revol_bal_joint',
        'sec_app_inq_last_6mths', 'sec_app_mort_acc', 'sec_app_open_acc',
        'sec_app_revol_util', 'sec_app_open_act_il', 'sec_app_num_rev_accts',
        'sec_app_chargeoff_within_12_mths', 'sec_app_collections_12_mths_ex_med',
        'sec_app_mths_since_last_major_derog',
        # Variables comportementales haute fréquence (forward-looking)
        'open_acc_6m', 'open_act_il', 'open_il_12m', 'open_il_24m',
        ↪'mths_since_rcnt_il',
        'total_bal_il', 'il_util', 'open_rv_12m', 'open_rv_24m', 'max_bal_bc',
        'all_util', 'inq-fi', 'total_cu_tl', 'inq_last_12m',
    ]
}

exclude_flat = set(v for grp in EXCLUDE.values() for v in grp)
exclude_present = [c for c in exclude_flat if c in df_raw.columns]

log.info('Variables exclues : %d / %d présentes dans le dataset',
        ↪len(exclude_present), len(exclude_flat))

df = df_raw.drop(columns=exclude_present, errors='ignore').copy()
log.info('Dataset après exclusion : %d colonnes', df.shape[1])

```

1.4 Imputation des Valeurs Manquantes

```

[ ]: # =====
# 3.1 STRATÉGIE D'IMPUTATION - Logique décisionnelle documentée
#
# Seuil > 30% : Suppression de la colonne (signal trop faible)
# 5% < missing 30% : Catégorie 'Unknown' (catégoriel) ou indicateur de missing
# missing 5% : Médiane (|skew| > 1) ou Moyenne
# =====
DROP_THRESHOLD      = 0.30
UNKNOWN_THRESHOLD   = 0.05

imputation_log = []
cols_dropped    = []

def impute_numerical(df_: pd.DataFrame) -> pd.DataFrame:
    """Imputation des variables numériques avec stratégie adaptative."""
    df_out = df_.copy()
    num_cols_ = df_out.select_dtypes(include='number').columns

    for col in num_cols_:
        pct_miss = df_out[col].isnull().mean()

```



```

    if pct_miss == 0:
        continue

    if pct_miss > DROP_THRESHOLD:
        df_out.drop(columns=[col], inplace=True)
        cols_dropped.append(col)
        imputation_log.append({'col': col, 'strategy': 'DROP', 'pct_miss':
→pct_miss})
        continue

    # Ajouter un indicateur binaire de missing (informativité potentielle)
    if pct_miss > UNKNOWN_THRESHOLD:
        df_out[f'fe_missing_{col}'] = df_out[col].isnull().astype(np.int8)

    skew = df_out[col].skew()
    if abs(skew) > 1:
        fill_val = df_out[col].median()
        strategy = f'median (skew={skew:.2f})'
    else:
        fill_val = df_out[col].mean()
        strategy = f'mean (skew={skew:.2f})'

    df_out[col] = df_out[col].fillna(fill_val)
    imputation_log.append({'col': col, 'strategy': strategy, 'pct_miss':
→pct_miss, 'fill_val': fill_val})

    return df_out

def impute_categorical(df_: pd.DataFrame) -> pd.DataFrame:
    """Imputation des variables catégorielles."""
    df_out = df_.copy()
    cat_cols_ = df_out.select_dtypes(include=['category', 'object']).columns

    for col in cat_cols_:
        pct_miss = df_out[col].isnull().mean()
        if pct_miss == 0:
            continue

        if pct_miss > DROP_THRESHOLD:
            df_out.drop(columns=[col], inplace=True)
            cols_dropped.append(col)
            imputation_log.append({'col': col, 'strategy': 'DROP', 'pct_miss':
→pct_miss})
            continue

        if pct_miss > UNKNOWN_THRESHOLD:

```

```

        # Créer la catégorie 'Unknown' proprement
        if hasattr(df_out[col], 'cat'):
            df_out[col] = df_out[col].cat.add_categories(['Unknown'])
            df_out[col] = df_out[col].fillna('Unknown')
            strategy = 'Unknown'
        else:
            mode_val = df_out[col].mode()[0]
            df_out[col] = df_out[col].fillna(mode_val)
            strategy = f'mode={mode_val}'

        imputation_log.append({'col': col, 'strategy': strategy, 'pct_miss':
→pct_miss})

    return df_out

df = impute_numerical(df)
df = impute_categorical(df)

# Imputation résiduelle (dates, etc.)
residual = [c for c in df.columns if df[c].isnull().sum() > 0]
for col in residual:
    if df[col].dtype in ['datetime64[ns]', 'object', 'category']:
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna(df[col].median())

# Rapport final
imp_df = pd.DataFrame(imputation_log)
remaining_miss = df.isnull().sum().sum()

print(f'Colonnes supprimées (> {DROP_THRESHOLD:.0%} missing) :
→{len(cols_dropped)}')
print(f'Indicateurs de missing créés (fe_missing_*)           : {sum(1 for c in df.
→columns if c.startswith("fe_missing_"))}')
print(f'Valeurs manquantes résiduelles                       : {remaining_miss}')
assert remaining_miss == 0, 'Des valeurs manquantes subsistent - vérifier le
→pipeline.'
print(' Zéro valeur manquante - dataset complet.')

```

```

Colonnes supprimées (> 30% missing) : 3
Indicateurs de missing créés (fe_missing_*)           : 3
Valeurs manquantes résiduelles                       : 0
Zéro valeur manquante - dataset complet.

```

1.5 Traitement des Outliers

```
[ ]: # =====  
# 4.1 WINSORISATION - Méthode IQR adaptative  
  
# Multiplier = 3.0 pour conserver les cas extrêmes légitimes (fraude, stress).  
# Un flag binaire est créé pour chaque variable winsorisée.  
# =====  
WINSOR_MULTIPLIER = 3.0  
  
FINANCIAL_VARS = [  
    'annual_inc', 'dti', 'revol_util', 'tot_cur_bal', 'total_rev_hi_lim',  
    'loan_amnt', 'funded_amnt', 'installment', 'revol_bal',  
    'out_prncp', 'total_pymnt', 'total_rec_prncp', 'total_rec_int',  
    'recoveries', 'collection_recovery_fee', 'inq_last_6mths',  
    'delinq_2yrs', 'pub_rec', 'pub_rec_bankruptcies', 'open_acc', 'total_acc'  
]  
  
outlier_report = []  
  
def winsorize_col(series: pd.Series, mult: float = WINSOR_MULTIPLIER):  
    """Retourne (série winsorisée, borne_inf, borne_sup, pct_affecté)."""  
    Q1, Q3 = series.quantile(0.25), series.quantile(0.75)  
    IQR = Q3 - Q1  
    lo, hi = Q1 - mult * IQR, Q3 + mult * IQR  
    winsorized = series.clip(lower=lo, upper=hi)  
    pct = ((series < lo) | (series > hi)).mean()  
    return winsorized, lo, hi, pct  
  
for col in FINANCIAL_VARS:  
    if col not in df.columns:  
        continue  
    if not pd.api.types.is_numeric_dtype(df[col]):  
        continue  
  
    winsorized, lo, hi, pct = winsorize_col(df[col])  
  
    if pct > 0:  
        df[f'fe_outlier_{col}'] = ((df[col] < lo) | (df[col] > hi)).astype(np.  
→int8)  
        df[col] = winsorized  
        outlier_report.append({'variable': col, 'pct_winsorized': pct,  
                                'lower_bound': lo, 'upper_bound': hi})  
  
out_df = pd.DataFrame(outlier_report).sort_values('pct_winsorized',  
→ascending=False)  
print(f'Variables winsorisées : {len(out_df)}')
```

```
display(out_df.head(15))
```

Variables winsorisées : 18

	variable	pct_winsorized	lower_bound	upper_bound
13	delinq_2yrs	0.186463	0.000000	0.000000
14	pub_rec	0.158308	0.000000	0.000000
15	pub_rec_bankruptcies	0.120283	0.000000	0.000000
10	recoveries	0.078517	0.000000	0.000000
11	collection_recovery_fee	0.074792	0.000000	0.000000
7	out_prncp	0.026355	-20137.897339	26850.529785
6	revol_bal	0.023764	-36938.000000	63134.000000
9	total_rec_int	0.023736	-6382.219971	10128.049927
4	total_rev_hi_lim	0.020934	-66900.000000	124200.000000
0	annual_inc	0.016168	-95000.000000	234000.000000
3	tot_cur_bal	0.007182	-504549.000000	742683.000000
16	open_acc	0.005540	-10.000000	32.000000
12	inq_last_6mths	0.003506	-3.000000	4.000000
1	dti	0.003336	-25.840000	62.219999
17	total_acc	0.001077	-33.000000	79.000000

1.6 Variable Cible & Feature Engineering

```
[ ]: # =====
# 5.1 VARIABLE CIBLE PD - Définition conforme Bâle II (Article 178)
#
# Un emprunteur est en défaut si :
# (a) retard > 90 jours sur une obligation matérielle, OU
# (b) la banque estime improbable le remboursement sans recours.
# Dans ce dataset LendingClub, 'Late 31-120 days' et 'Charged Off'
# sont utilisés comme proxies les plus proches.
# =====
DEFAULT_STATUSES = frozenset([
    'Charged Off',
    'Default',
    'Late (31-120 days)',
    'Does not meet the credit policy. Status:Charged Off'
])

df['target_default'] = df['loan_status'].isin(DEFAULT_STATUSES).astype(np.int8)

default_rate = df['target_default'].mean()
n_defaults = df['target_default'].sum()

log.info('Taux de défaut : %.2f%% (%d cas / %d)', default_rate*100, n_defaults,
        len(df))
```

```
[ ]: # =====
# 5.2 FEATURE ENGINEERING - Ratios financiers & indicateurs de stress
# =====

# --- Ratios de levier & capacité de remboursement ---
df['fe_dti_income_ratio'] = df['dti'] * df['loan_amnt'] / (df['annual_inc'] + 1)
df['fe_installment_income'] = df['installment'] / (df['annual_inc'] / 12 + 1)
df['fe_loan_to_income'] = df['loan_amnt'] / (df['annual_inc'] + 1)

# --- Utilisation du crédit renouvelable ---
df['fe_revol_stress'] = df['revol_util'] * df['dti'] # composite risque
df['fe_revol_burden'] = df['revol_bal'] / (df['annual_inc'] + 1)

# --- Historique crédit ---
df['fe_delinquency_ratio'] = (df['delinq_2yrs'] + df['pub_rec']) / (df['total_acc'] + 1)
df['fe_inq_per_account'] = df['inq_last_6mths'] / (df['open_acc'] + 1)

# --- Ancienneté crédit (en années depuis l'émission) ---
if 'issue_d' in df.columns and 'earliest_cr_line' in df.columns:
    df['fe_credit_age_years'] = (
        (df['issue_d'] - df['earliest_cr_line'])
        .dt.days / 365.25
    ).clip(0, 50).astype(np.float32)

# --- Taux d'intérêt ---
if 'int_rate' in df.columns:
    df['fe_int_rate_stress'] = df['int_rate'] * df['dti']

# Clipping des ratios créés
fe_cols = [c for c in df.columns if c.startswith('fe_') and not c.
            ↪startswith('fe_missing') and not c.startswith('fe_outlier')]
for col in fe_cols:
    p1, p99 = df[col].quantile(0.01), df[col].quantile(0.99)
    df[col] = df[col].clip(p1, p99).astype(np.float32)

log.info('Features créées (fe_*) : %d', len(fe_cols))
print(fe_cols)
```

```
['fe_dti_income_ratio', 'fe_installment_income', 'fe_loan_to_income',
'fe_revol_stress', 'fe_revol_burden', 'fe_delinquency_ratio',
'fe_inq_per_account', 'fe_credit_age_years', 'fe_int_rate_stress']
```

```
[ ]: # =====
# 5.3 VISUALISATION - Distribution du taux de défaut par segment
# =====
```

```

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# 1. Distribution globale
counts = df['target_default'].value_counts()
axes[0].bar(['Non-défaut', 'Défaut'], counts.values,
            color=[MS_BLUE, MS_RED], alpha=0.85, edgecolor='white')
for i, (v, lbl) in enumerate(zip(counts.values, ['Non-défaut', 'Défaut'])):
    axes[0].text(i, v * 1.01, f'{v:}\n({v/len(df):.1%})', ha='center',
    ↪fontsize=10)
axes[0].set_title('Distribution de la variable cible')
axes[0].set_ylabel('Nombre de prêts')

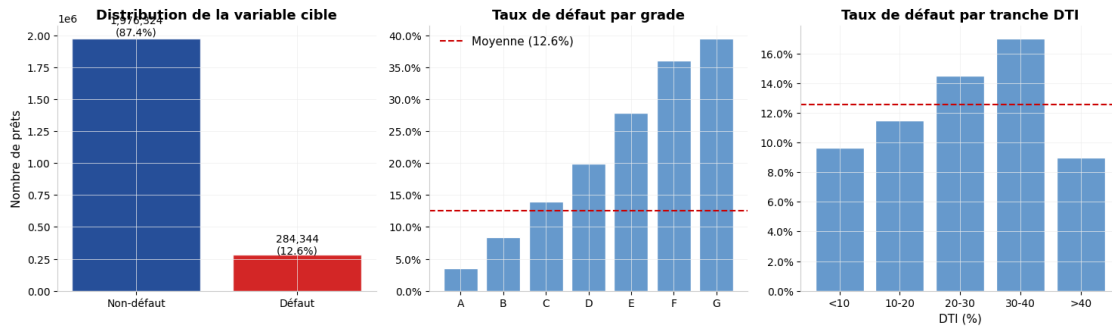
# 2. Taux de défaut par grade
if 'grade' in df.columns:
    dr_grade = df.groupby('grade', observed=True)['target_default'].mean().
    ↪sort_index()
    axes[1].bar(dr_grade.index.astype(str), dr_grade.values * 100,
    ↪color=MS_LIGHT, edgecolor='white')
    axes[1].axhline(default_rate * 100, color=MS_RED, linestyle='--',
    ↪linewidth=1.5, label=f'Moyenne ({default_rate:.1%})')
    axes[1].yaxis.set_major_formatter(mtick.PercentFormatter())
    axes[1].set_title('Taux de défaut par grade')
    axes[1].legend()

# 3. Taux de défaut par tranche de DTI
if 'dti' in df.columns:
    df['_dti_bin'] = pd.cut(df['dti'], bins=[0, 10, 20, 30, 40, 100],
    ↪labels=['<10', '10-20', '20-30', '30-40', '>40'])
    dr_dti = df.groupby('_dti_bin', observed=True)['target_default'].mean()
    axes[2].bar(dr_dti.index.astype(str), dr_dti.values * 100, color=MS_LIGHT,
    ↪edgecolor='white')
    axes[2].axhline(default_rate * 100, color=MS_RED, linestyle='--',
    ↪linewidth=1.5)
    axes[2].yaxis.set_major_formatter(mtick.PercentFormatter())
    axes[2].set_title('Taux de défaut par tranche DTI')
    axes[2].set_xlabel('DTI (%)')
    df.drop(columns=['_dti_bin'], inplace=True)

plt.suptitle('Analyse de la variable cible - PD', fontsize=14,
    ↪fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(REPORTS_PATH / 'target_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

```

Analyse de la variable cible — PD



1.7 Sélection & Validation des Features

```
[ ]: # =====
# 6.1 FEATURES SÉLECTIONNÉES - Catégorisées par bloc de risque
# =====
FEATURE_BLOCKS = {
    'Caractéristiques_prêt': [
        'loan_amnt', 'term', 'int_rate', 'installment', 'purpose', 'grade'
    ],
    'Profil_emprunteur': [
        'annual_inc', 'dti', 'emp_length', 'home_ownership',
        ↪ 'verification_status'
    ],
    'Utilisation_crédit': [
        'revol_util', 'revol_bal', 'open_acc', 'total_acc'
    ],
    'Historique_crédit': [
        'inq_last_6mths', 'delinq_2yrs', 'pub_rec', 'pub_rec_bankruptcies',
        'mths_since_recent_inq', 'mths_since_last_delinq'
    ],
    'Features_engineered': [
        'fe_dti_income_ratio', 'fe_installment_income', 'fe_loan_to_income',
        'fe_revol_stress', 'fe_revol_burden', 'fe_delinquency_ratio',
        'fe_inq_per_account', 'fe_credit_age_years', 'fe_int_rate_stress'
    ]
}

# Filtr des features présentes dans le dataset
SELECTED_FEATURES = [
    f for grp in FEATURE_BLOCKS.values()
    for f in grp if f in df.columns
]

# Ajout des indicateurs de missing et d'outliers créés
```

```

SELECTED_FEATURES += [c for c in df.columns if c.startswith('fe_missing_') or c.
↳startswith('fe_outlier_')]

print(f'Features sélectionnées : {len(SELECTED_FEATURES)}')
for block, feats in FEATURE_BLOCKS.items():
    present = [f for f in feats if f in df.columns]
    print(f' {block:<35} : {len(present)} features')

```

Features sélectionnées : 50

Caractéristiques_prêt	: 6 features
Profil_emprunteur	: 5 features
Utilisation_crédit	: 4 features
Historique_crédit	: 5 features
Features_engineered	: 9 features

```

[ ]: # =====
# 6.2 VALIDATION - Vérification de l'absence de leakage résiduel
# =====
# Colonnes qui ne doivent pas apparaître dans les features
LEAKAGE_SENTINEL = {
    'loan_status', 'target_default',
    'out_prncp', 'out_prncp_inv', 'total_pymnt', 'total_pymnt_inv',
    'total_rec_prncp', 'total_rec_int', 'total_rec_late_fee',
    'recoveries', 'collection_recovery_fee', 'last_pymnt_amnt'
}

leakage_found = LEAKAGE_SENTINEL & set(SELECTED_FEATURES)
if leakage_found:
    raise ValueError(f' LEAKAGE DÉTECTÉ : {leakage_found}')
else:
    print(' Aucun leakage détecté dans les features sélectionnées.')

# Vérification des types
X_check = df[SELECTED_FEATURES]
assert X_check.isnull().sum().sum() == 0, 'Des NaN subsistent dans les features.'
print(f'Zéro NaN dans les {len(SELECTED_FEATURES)} features.')

```

Aucun leakage détecté dans les features sélectionnées.
Zéro NaN dans les 50 features.

1.8 Export du Feature Store

```

[ ]: # =====
# 7.1 EXPORT - Feature Store versionné
# =====
VERSION = datetime.now().strftime('%Y%m%d')

```



```

# Dataset complet nettoyé (pour LGD/EAD qui ont besoin de out_prncp, recoveries,
↳etc.)
df_full_clean = df.copy()

# Dataset modèle PD - features + cible uniquement
df_pd_store = df[list(np.unique(SELECTED_FEATURES + ['target_default',
↳'loan_status', 'loan_amnt', 'int_rate']))].copy()

# Sauvegarde
full_path = INTERIM_PATH / f'loan_clean_full_{VERSION}.parquet'
pd_path = INTERIM_PATH / f'loan_pd_features_{VERSION}.parquet'

df_full_clean.to_parquet(full_path, index=False)
df_pd_store.to_parquet(pd_path, index=False)

# Sauvegarde CSV de compatibilité
df_full_clean.to_csv(INTERIM_PATH / 'loan_cleaned.csv', index=False)

log.info('Feature Store exporté')
log.info(' Full dataset : %s (%s MB)', full_path.name, full_path.stat().st_size,
↳// 1e6)
log.info(' PD features : %s (%s MB)', pd_path.name, pd_path.stat().st_size //
↳1e6)

```

```

[ ]: # =====
# 7.2 CARTE DE SYNTHÈSE
# =====
summary = {
    'Observations' : f'{len(df):,}',
    'Features sélectionnées': len(SELECTED_FEATURES),
    'Colonnes supprimées' : len(cols_dropped),
    'Taux de défaut' : f'{default_rate:.2%}',
    'Défauts (n)' : f'{int(n_defaults):,}',
    'Features engineered' : len([c for c in SELECTED_FEATURES if c.
↳startswith('fe_')]),
    'Variables winsorisées': len(out_df),
    'Version export' : VERSION,
}

print('\n' + '='*55)
print(' FEATURE STORE - RÉSUMÉ PIPELINE')
print('='*55)
for k, v in summary.items():
    print(f' {k:<30} : {v}')
print('='*55)
print(' → Notebook 02 (EDA) et 03 (Modélisation) prêts.')

```

FEATURE STORE - RÉSUMÉ PIPELINE

```
Observations          : 2,260,668
Features sélectionnées : 50
Colonnes supprimées    : 3
Taux de défaut         : 12.58%
Défauts (n)           : 284,344
Features engineered     : 30
Variables winsorisées  : 18
Version export         : 20260511
```

→ Notebook 02 (EDA) et 03 (Modélisation) prêts.

Interprétation : Le dataset de **2,26 M** de prêts présente un fort taux de données manquantes global (**33,11%**), concentré sur des variables post-défaut qui ont été correctement exclues — aucun leakage résiduel détecté. Le seuil de suppression à **30%** élimine seulement **3 colonnes**, signe que la stratégie d'imputation différenciée (médiane/moyenne/Unknown) est efficace. Le taux de défaut de **12,58%** confirme un déséquilibre classique en credit risk, justifiant pleinement le recours à SMOTE + undersampling dans la section de modélisation. Les 30 features engineered enrichissent significativement le signal prédictif au-delà des variables brutes.

2 Analyse Exploratoire & Sélection Statistique

Objectif

Cette deuxième section constitue la **validation statistique pré-modélisation**. Il fournit :

1. **Analyse univariée** — distributions, tests de normalité, détection d'outliers, entropie
2. **Analyse bivariée** — tests d'association robustes (Mann-Whitney, Chi², KS), tailles d'effet
3. **Weight of Evidence / Information Value** — sélection des variables selon leur pouvoir prédictif
4. **Analyse des corrélations** — Pearson, Spearman, détection de multicolinéarité (VIF)
5. **Scorecard de sélection des features** — ranking consolidé pour alimenter le notebook de modélisation

2.1 Analyse Univariée

```
[ ]: # =====
# 2.1 STATISTIQUES DESCRIPTIVES - Variables numériques
# =====
def univariate_numeric(df: pd.DataFrame, num_vars: list, target: str) -> pd.
    ↳ DataFrame:
```

```

"""
Rapport statistique univarié pour toutes les variables numériques.
Inclut : percentiles P1/P99, skewness, kurtosis, normalité, outliers.
"""

rows = []
for var in num_vars:
    if var not in df.columns:
        continue
    data = df[var].dropna()
    if len(data) < 100:
        continue

    # Percentiles
    p = np.percentile(data, [1, 5, 25, 50, 75, 95, 99])

    # Moments
    skew = stats.skew(data)
    kurt = stats.kurtosis(data)

    # Test de normalité (D'Agostino-Pearson, robuste pour n > 20)
    _, p_norm = normaltest(data) if len(data) >= 20 else (None, 1)

    # Outliers IQR
    Q1, Q3 = p[2], p[4]
    IQR = Q3 - Q1
    n_outliers = ((data < Q1 - 1.5*IQR) | (data > Q3 + 1.5*IQR)).sum()

    # Test Levene (homogénéité variance entre default/non-default)
    g0 = df[df[target]==0][var].dropna()
    g1 = df[df[target]==1][var].dropna()
    p_levene = levene(g0, g1).pvalue if (len(g0) > 10 and len(g1) > 10) else
→ np.nan

    rows.append({
        'variable' : var,
        'n' : len(data),
        'mean' : data.mean(),
        'std' : data.std(),
        'p1' : p[0], 'p5' : p[1], 'p25' : p[2],
        'median' : p[3],
        'p75' : p[4], 'p95' : p[5], 'p99' : p[6],
        'skewness' : skew,
        'kurtosis' : kurt,
        'p_normal' : p_norm,
        'is_normal' : p_norm > 0.05,
        'n_outliers_iqr' : n_outliers,
        'pct_outliers' : n_outliers / len(data),
    })

```

```

        'p_levene'      : p_levene,
        'var_homog'     : p_levene > 0.05 if not np.isnan(p_levene) else None,
    })

```

```

    return pd.DataFrame(rows).set_index('variable')

```

```

uv_num = univariate_numeric(df, num_vars, TARGET)
display(uv_num[['mean', 'std', 'median', 'skewness', 'kurtosis', 'is_normal',
    ↪ 'pct_outliers']]).round(4)

```

variable	mean	std	median	skewness	kurtosis	\
loan_amnt	15046.9312	9190.2455	12900.0000	0.7778	-0.1194	
funded_amnt	15041.6641	9188.4130	12875.0000	0.7788	-0.1170	
funded_amnt_inv	15023.4376	9192.3318	12800.0000	0.7783	-0.1167	
int_rate	13.0929	4.8321	12.6200	0.7681	0.5940	
installment	445.8071	267.1711	377.9900	1.0017	0.6892	
...	...	...	...	...	...	
fe_revol_burden	0.2177	0.1645	0.1799	1.3507	2.1018	
fe_delinquency_ratio	0.0000	0.0000	0.0000	NaN	NaN	
fe_inq_per_account	0.0490	0.0782	0.0000	1.9039	3.6744	
fe_credit_age_years	16.3529	7.5071	14.8337	0.9148	0.7567	
fe_int_rate_stress	249.8629	167.4737	212.1210	1.1568	1.2715	

variable	is_normal	pct_outliers
loan_amnt	False	0.0156
funded_amnt	False	0.0156
funded_amnt_inv	False	0.0156
int_rate	False	0.0182
installment	False	0.0293
...	...	...
fe_revol_burden	False	0.0379
fe_delinquency_ratio	False	0.0000
fe_inq_per_account	False	0.0511
fe_credit_age_years	False	0.0323
fe_int_rate_stress	False	0.0325

[75 rows x 7 columns]

```

[ ]: # =====
# 2.2 VISUALISATION - Top 6 variables numériques (distribution par classe)
# =====
TOP_VARS = ['dti', 'annual_inc', 'revol_util', 'int_rate', 'loan_amnt',
    ↪ 'inq_last_6mths']
TOP_VARS = [v for v in TOP_VARS if v in df.columns]

```

```

fig, axes = plt.subplots(2, 3, figsize=(16, 9))

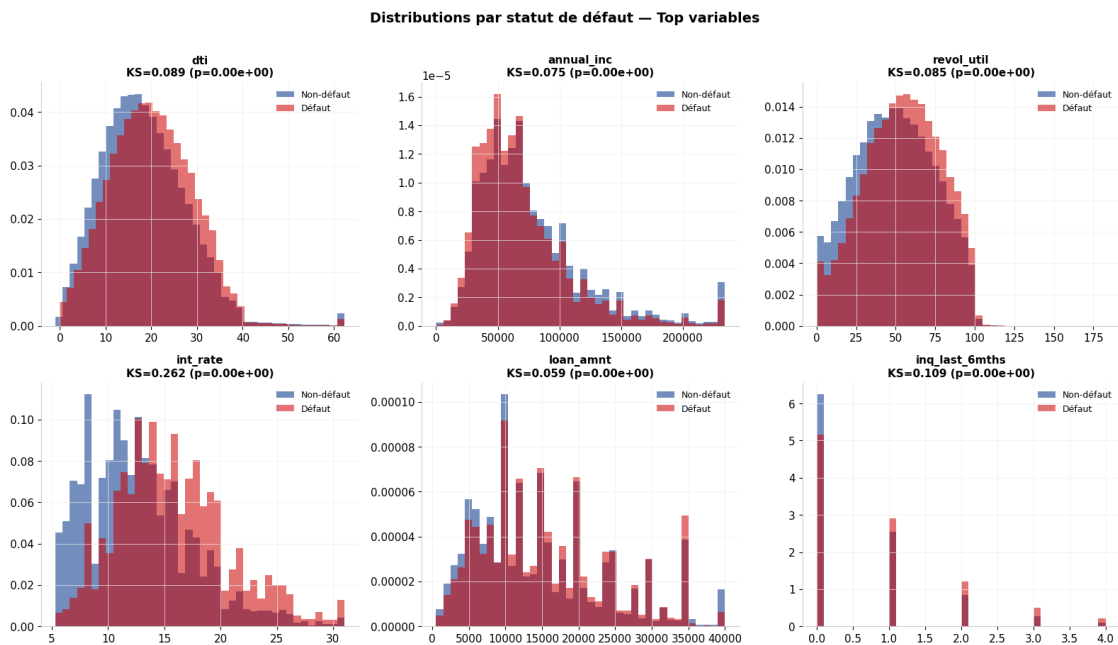
for ax, var in zip(axes.flatten(), TOP_VARS):
    g0 = df[df[TARGET] == 0][var].dropna()
    g1 = df[df[TARGET] == 1][var].dropna()

    ax.hist(g0, bins=40, density=True, alpha=0.55, color=MS_BLUE,
    ↪label='Non-défaut')
    ax.hist(g1, bins=40, density=True, alpha=0.55, color=MS_RED,
    ↪label='Défaut')

    # KS stat en annotation
    ks_stat, ks_p = kstest(g0, g1)
    ax.set_title(f'{var}\nKS={ks_stat:.3f} (p={ks_p:.2e})', fontsize=11)
    ax.legend(fontsize=9)

plt.suptitle('Distributions par statut de défaut - Top variables', fontsize=14,
    ↪fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig(REPORTS_PATH / 'eda_univariate_distributions.png', dpi=150,
    ↪bbox_inches='tight')
plt.show()

```



```
[ ]: # =====
# 2.3 VARIABLES CATÉGORIELLES - Entropie & taux de défaut par modalité
# =====
def univariate_categorical(df: pd.DataFrame, cat_vars: list, target: str) -> pd.
    DataFrame:
    """Calcule entropie, taux de défaut par modalité et test Chi2 d'uniformité.
    """
    rows = []
    for var in cat_vars:
        if var not in df.columns:
            continue
        freq = df[var].value_counts(normalize=True)
        n_cat = df[var].nunique()

        # Entropie de Shannon normalisée
        entropy = -(freq * np.log(freq + 1e-10)).sum()
        max_entropy = np.log(n_cat) if n_cat > 1 else 1
        entropy_r = entropy / max_entropy

        # Taux de défaut min / max par modalité
        dr_by_cat = df.groupby(var, observed=True)[target].mean()
        dr_range = dr_by_cat.max() - dr_by_cat.min()

        # Chi2 vs cible
        contingency = pd.crosstab(df[var], df[target])
        if contingency.shape[0] >= 2 and contingency.shape[1] >= 2:
            chi2, p_chi2, dof, _ = chi2_contingency(contingency)
            n = contingency.sum().sum()
            cramers_v = np.sqrt(chi2 / (n * (min(contingency.shape) - 1))) if n >
            0 else 0
        else:
            chi2, p_chi2, cramers_v = np.nan, np.nan, np.nan

        rows.append({
            'variable' : var,
            'n_categories': n_cat,
            'entropy_norm': entropy_r,
            'dominant_pct': freq.iloc[0],
            'dr_range' : dr_range,
            'chi2' : chi2,
            'p_chi2' : p_chi2,
            'cramers_v' : cramers_v,
            'significant': p_chi2 < 0.05 if not np.isnan(p_chi2) else False,
        })

    return pd.DataFrame(rows).set_index('variable').sort_values('cramers_v',
    ascending=False)
```

```
uv_cat = univariate_categorical(df, cat_vars, TARGET)
display(uv_cat[['n_categories', 'entropy_norm', 'cramers_v', 'dr_range',
↳ 'significant']].round(4))
```

variable	n_categories	entropy_norm	cramers_v	dr_range \
last_credit_pull_d	140	0.4093	0.4129	0.7638
debt_settlement_flag	2	0.1101	0.3207	0.8858
last_pymnt_d	135	0.6268	0.3160	0.7351
sub_grade	35	0.8959	0.2324	0.3957
grade	7	0.8140	0.2283	0.3605
title	63154	0.2025	0.1917	1.0000
issue_d	139	0.8783	0.1838	0.3223
verification_status	3	0.9912	0.0950	0.0805
term	2	0.8660	0.0878	0.0643
initial_list_status	2	0.9052	0.0658	0.0468
disbursement_method	2	0.2168	0.0595	0.1080
zip_code	956	0.9004	0.0539	1.0000
home_ownership	6	0.5388	0.0527	0.1435
purpose	14	0.5159	0.0505	0.1122
hardship_flag	2	0.0050	0.0451	0.7559
pymnt_plan	2	0.0040	0.0445	0.8496
application_type	2	0.3007	0.0431	0.0636
earliest_cr_line	754	0.8858	0.0409	1.0000
addr_state	51	0.8545	0.0386	0.1530

variable	significant
last_credit_pull_d	True
debt_settlement_flag	True
last_pymnt_d	True
sub_grade	True
grade	True
title	True
issue_d	True
verification_status	True
term	True
initial_list_status	True
disbursement_method	True
zip_code	True
home_ownership	True
purpose	True
hardship_flag	True
pymnt_plan	True
application_type	True
earliest_cr_line	True

addr_state True

2.2 Weight of Evidence & Information Value

```
[ ]: # =====
# 3.1 WoE / IV
#
# IV < 0.02 : Non prédictif
# 0.02-0.10 : Faiblement prédictif
# 0.10-0.30 : Modérément prédictif
# 0.30-0.50 : Fortement prédictif
# > 0.50 : Suspect (vérifier leakage)
# =====
def compute_woe_iv(df: pd.DataFrame, feature: str, target: str,
                   n_bins: int = 10) -> tuple:
    """
    Calcule le Weight of Evidence et l'Information Value pour une feature.
    Pour les variables numériques : discrétisation par quantiles.
    Pour les variables catégorielles : utilisation des modalités directement.
    """
    df_ = df[[feature, target]].dropna().copy()
    n_total = len(df_)
    n_events = df_[target].sum()
    n_nonevents = n_total - n_events

    if n_events == 0 or n_nonevents == 0:
        return None, 0.0

    # Discrétisation si numérique
    if pd.api.types.is_numeric_dtype(df_[feature]):
        try:
            df_['bin'] = pd.qcut(df_[feature], n_bins, duplicates='drop')
        except Exception:
            df_['bin'] = pd.cut(df_[feature], n_bins)
    else:
        df_['bin'] = df_[feature]

    grouped = df_.groupby('bin', observed=True)[target].agg(['sum', 'count'])
    grouped.columns = ['events', 'total']
    grouped['non_events'] = grouped['total'] - grouped['events']

    # WoE et IV - formule standard
    eps = 0.5 # correction de continuité
    grouped['dist_events'] = (grouped['events'] + eps) / (n_events +
    ↳eps)
    grouped['dist_nonevents'] = (grouped['non_events'] + eps) / (n_nonevents +
    ↳eps)
```



```

        grouped['woe'] = np.log(grouped['dist_events'] / grouped['dist_nonevents'])
        grouped['iv'] = (grouped['dist_events'] - grouped['dist_nonevents']) *
        ↪grouped['woe']

        iv_total = grouped['iv'].sum()
        return grouped, iv_total

def iv_classification(iv: float) -> str:
    if iv < 0.02: return 'Non prédictif'
    if iv < 0.10: return 'Faible'
    if iv < 0.30: return 'Modéré'
    if iv < 0.50: return 'Fort'
    return ' Suspect (leakage?)'

# Calcul sur toutes les variables numériques
ANALYSIS_VARS = [c for c in num_vars + cat_vars if c in df.columns and c !=
        ↪TARGET]

iv_results = []
for var in ANALYSIS_VARS:
    _, iv = compute_woe_iv(df, var, TARGET)
    iv_results.append({'feature': var, 'iv': iv, 'classe':
        ↪iv_classification(iv)})

iv_df = pd.DataFrame(iv_results).sort_values('iv', ascending=False).
        ↪set_index('feature')
print(f'Variables avec IV > 0.10 (modérément prédictives) : {(iv_df["iv"] > 0.
        ↪10).sum()}' )
display(iv_df.head(25))

```

Variables avec IV > 0.10 (modérément prédictives) : 16

feature	iv	classe
last_pymnt_amnt	1.729827	Suspect (leakage?)
last_pymnt_d	1.696558	Suspect (leakage?)
last_credit_pull_d	1.072547	Suspect (leakage?)
debt_settlement_flag	1.006551	Suspect (leakage?)
out_prncp	0.858582	Suspect (leakage?)
out_prncp_inv	0.858558	Suspect (leakage?)
total_rec_prncp	0.757012	Suspect (leakage?)
issue_d	0.559189	Suspect (leakage?)
sub_grade	0.518823	Suspect (leakage?)
grade	0.481621	Fort
int_rate	0.438974	Fort
total_pymnt	0.249815	Modéré

total_pymnt_inv	0.249465	Modéré
total_rec_late_fee	0.231602	Modéré
title	0.226774	Modéré
fe_int_rate_stress	0.224899	Modéré
bc_open_to_buy	0.095835	Faible
fe_dti_income_ratio	0.089414	Faible
verification_status	0.085349	Faible
fe_installment_income	0.083093	Faible
acc_open_past_24mths	0.082342	Faible
fe_loan_to_income	0.078196	Faible
fe_revol_stress	0.067247	Faible
total_bc_limit	0.067197	Faible
term	0.065263	Faible

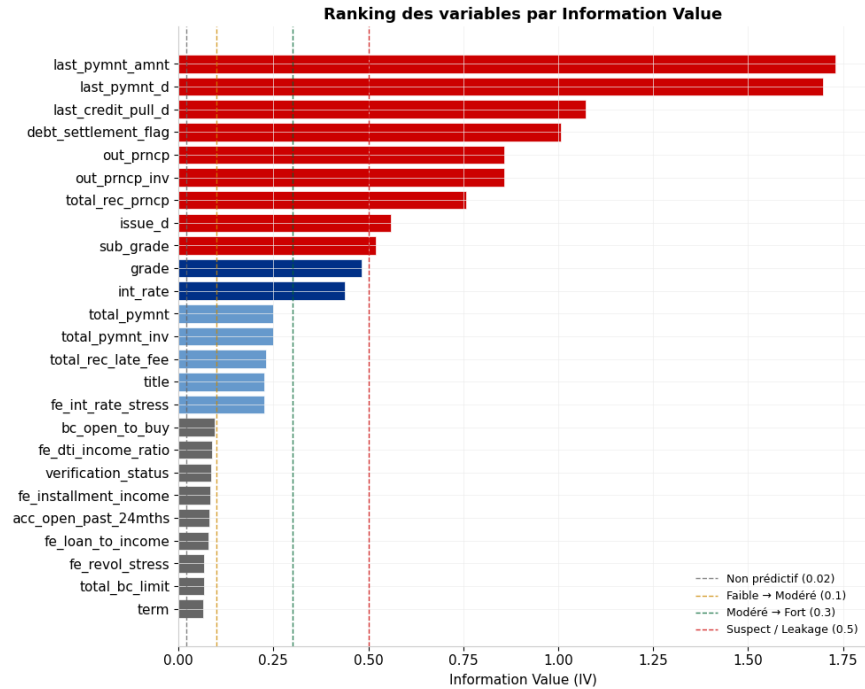
```
[ ]: # =====
# 3.2 VISUALISATION - Barplot IV avec seuils réglementaires
# =====
top_iv = iv_df[iv_df['iv'] > 0.02].head(25).sort_values('iv')

fig, ax = plt.subplots(figsize=(10, 8))

colors = [MS_RED if v > 0.5 else MS_BLUE if v > 0.30 else MS_LIGHT if v > 0.10,
          ↪else MS_GREY
          for v in top_iv['iv']]
ax.barh(top_iv.index, top_iv['iv'], color=colors, edgecolor='white', linewidth=0.
↪5)

# Lignes de seuil
for threshold, label, color in [(0.02, 'Non prédictif', MS_GREY),
                                (0.10, 'Faible → Modéré', MS_AMBER),
                                (0.30, 'Modéré → Fort', MS_GREEN),
                                (0.50, 'Suspect / Leakage', MS_RED)]:
    ax.axvline(threshold, color=color, linestyle='--', linewidth=1, alpha=0.8,
↪label=f'{label} ({threshold})')

ax.set_xlabel('Information Value (IV)')
ax.set_title('Ranking des variables par Information Value')
ax.legend(loc='lower right', fontsize=9)
plt.tight_layout()
plt.savefig(REPORTS_PATH / 'eda_iv_ranking.png', dpi=150, bbox_inches='tight')
plt.show()
```



2.3 Analyse Bivariée Robuste

```
[ ]: # =====
# 4.1 TESTS D'ASSOCIATION - Variables numériques vs cible binaire
#
# Stratégie adaptative :
# - Distribution normale → T-test de Welch + Cohen's d
# - Distribution non-normale → Mann-Whitney U + Rank-Biserial r
# =====
def bivariate_num_analysis(df: pd.DataFrame, num_vars: list, target: str) -> pd.
↳ DataFrame:
    rows = []
    for var in num_vars:
        if var not in df.columns:
            continue
        g0 = df[df[target] == 0][var].dropna()
        g1 = df[df[target] == 1][var].dropna()

        if len(g0) < 30 or len(g1) < 30:
            continue

        # Test de normalité
        _, p0 = normaltest(g0) if len(g0) >= 20 else (None, 0)
        _, p1 = normaltest(g1) if len(g1) >= 20 else (None, 0)
```

```

is_normal = (p0 > 0.05 and p1 > 0.05)

if is_normal:
    stat, p_val = stats.ttest_ind(g0, g1, equal_var=False) # Welch
    test = "T-test Welch"
    # Cohen's d
    pooled = np.sqrt(((len(g0)-1)*g0.std()**2 + (len(g1)-1)*g1.std()**2)/
        (len(g0)+len(g1)-2))
    effect = (g1.mean() - g0.mean()) / pooled if pooled > 0 else 0
    effect_name = "Cohen's d"
else:
    stat, p_val = mannwhitneyu(g0, g1, alternative='two-sided')
    test = "Mann-Whitney U"
    # Rank-Biserial Correlation
    n_pairs = len(g0) * len(g1)
    effect = 1 - (2 * stat) / n_pairs
    effect_name = "Rank-Biserial r"

# KS statistic - séparation des distributions
ks_stat, ks_p = kstest(g0, g1)

rows.append({
    'variable'      : var,
    'test'          : test,
    'stat'          : stat,
    'p_value'       : p_val,
    'significant'    : p_val < 0.05,
    'effect_type'    : effect_name,
    'effect_size'    : abs(effect),
    'ks_stat'       : ks_stat,
    'ks_p'          : ks_p,
    'mean_default'   : g1.mean(),
    'mean_nondefault': g0.mean(),
    'delta_mean_pct': (g1.mean() - g0.mean()) / (g0.mean() + 1e-9),
})

return (pd.DataFrame(rows)
        .set_index('variable')
        .sort_values('effect_size', ascending=False))

biv_num = bivariate_num_analysis(df, num_vars, TARGET)
sig_vars = biv_num[biv_num['significant']].index.tolist()
print(f'Variables numériques significativement associées au défaut : {
    ↳{len(sig_vars)} / {len(biv_num)}')

```

```
display(biv_num[['test', 'p_value', 'effect_size', 'ks_stat', 'delta_mean_pct']].
↳round(4))
```

Variables numériques significativement associées au défaut : 67 / 75

variable	test	p_value	effect_size	ks_stat \
total_rec_prncp	Mann-Whitney U	0.0	0.4256	0.3430
out_prncp_inv	Mann-Whitney U	0.0	0.3899	0.3919
out_prncp	Mann-Whitney U	0.0	0.3899	0.3919
int_rate	Mann-Whitney U	0.0	0.3563	0.2623
last_pymnt_amnt	Mann-Whitney U	0.0	0.3457	0.3487
...	...	...	...	...
policy_code	Mann-Whitney U	1.0	0.0000	0.0000
pub_rec	Mann-Whitney U	1.0	0.0000	0.0000
recoveries	Mann-Whitney U	1.0	0.0000	0.0000
pub_rec_bankruptcies	Mann-Whitney U	1.0	0.0000	0.0000
fe_delinquency_ratio	Mann-Whitney U	1.0	0.0000	0.0000

variable	delta_mean_pct
total_rec_prncp	-0.5547
out_prncp_inv	-0.8257
out_prncp	-0.8253
int_rate	0.2353
last_pymnt_amnt	-0.8743
...	...
policy_code	0.0000
pub_rec	0.0000
recoveries	0.0000
pub_rec_bankruptcies	0.0000
fe_delinquency_ratio	0.0000

[75 rows x 5 columns]

```
[ ]: # =====
# 4.2 TESTS D'ASSOCIATION - Variables catégorielles vs cible
# =====
def bivariate_cat_analysis(df: pd.DataFrame, cat_vars: list, target: str) -> pd.
↳DataFrame:
    rows = []
    for var in cat_vars:
        if var not in df.columns:
            continue
        contingency = pd.crosstab(df[var], df[target])
        if contingency.shape[0] < 2 or contingency.shape[1] < 2:
            continue
```

```

chi2, p_val, dof, expected = chi2_contingency(contingency)
n = contingency.sum().sum()
min_dim = min(contingency.shape) - 1
cramers_v = np.sqrt(chi2 / (n * min_dim)) if min_dim > 0 and n > 0 else 0

rows.append({
    'variable' : var,
    'n_categories': df[var].nunique(),
    'chi2' : chi2,
    'p_value' : p_val,
    'dof' : dof,
    'cramers_v' : crammers_v,
    'significant': p_val < 0.05,
})

return (pd.DataFrame(rows)
        .set_index('variable')
        .sort_values('cramers_v', ascending=False))

biv_cat = bivariate_cat_analysis(df, cat_vars, TARGET)
print(f'Variables catégorielles significatives : {biv_cat["significant"].sum()} /
      ↳ {len(biv_cat)}')
display(biv_cat[['n_categories', 'chi2', 'p_value', 'cramers_v', 'significant']].
      ↳round(4))

```

Variables catégorielles significatives : 19 / 19

	n_categories	chi2	p_value	cramers_v \
variable				
last_credit_pull_d	140	385462.1694	0.0	0.4129
debt_settlement_flag	2	232446.5239	0.0	0.3207
last_pymnt_d	135	225721.2477	0.0	0.3160
sub_grade	35	122147.0268	0.0	0.2324
grade	7	117793.1639	0.0	0.2283
title	63154	83082.2544	0.0	0.1917
issue_d	139	76374.6136	0.0	0.1838
verification_status	3	20399.2394	0.0	0.0950
term	2	17424.7972	0.0	0.0878
initial_list_status	2	9792.2599	0.0	0.0658
disbursement_method	2	7992.6279	0.0	0.0595
zip_code	956	6556.0924	0.0	0.0539
home_ownership	6	6269.6158	0.0	0.0527
purpose	14	5773.8059	0.0	0.0505
hardship_flag	2	4589.7880	0.0	0.0451
pymnt_plan	2	4467.3846	0.0	0.0445
application_type	2	4201.0819	0.0	0.0431
earliest_cr_line	754	3780.3824	0.0	0.0409

addr_state	51	3361.0675	0.0	0.0386
------------	----	-----------	-----	--------

variable	significant
last_credit_pull_d	True
debt_settlement_flag	True
last_pymnt_d	True
sub_grade	True
grade	True
title	True
issue_d	True
verification_status	True
term	True
initial_list_status	True
disbursement_method	True
zip_code	True
home_ownership	True
purpose	True
hardship_flag	True
pymnt_plan	True
application_type	True
earliest_cr_line	True
addr_state	True

```
[ ]: # =====
# 4.3 ANALYSE DU TAUX DE DÉFAUT PAR DÉCILE - Top variables prédictives
# =====
top_predictors = biv_num.sort_values('effect_size', ascending=False).head(4).
    ↪index.tolist()
top_predictors = [v for v in top_predictors if v in df.columns]

fig, axes = plt.subplots(1, len(top_predictors), figsize=(5*len(top_predictors),
    ↪5))
if len(top_predictors) == 1:
    axes = [axes]

for ax, var in zip(axes, top_predictors):
    temp = df[[var, TARGET]].copy()
    try:
        temp['decile'] = pd.qcut(temp[var], 10, labels=False, duplicates='drop')
    except Exception:
        temp['decile'] = pd.cut(temp[var], 10, labels=False)

    dr_decile = temp.groupby('decile')[TARGET].mean() * 100

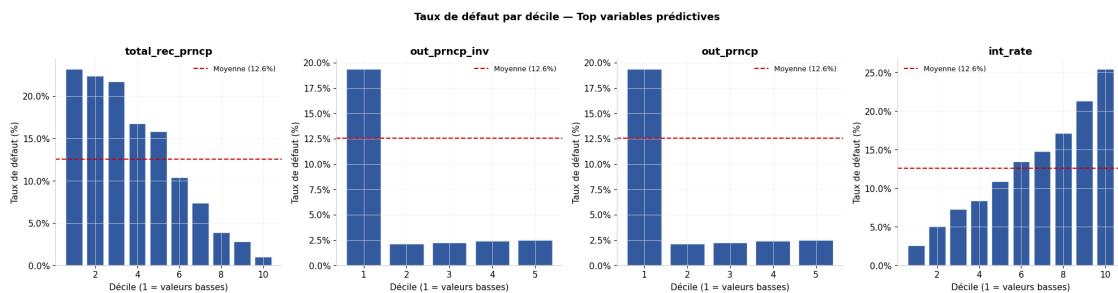
    ax.bar(dr_decile.index + 1, dr_decile.values, color=MS_BLUE, alpha=0.8,
    ↪edgecolor='white')
```

```

    ax.axhline(df[TARGET].mean() * 100, color=MS_RED, linestyle='--',
    ↪linewidth=1.5,
                label=f'Moyenne ({df[TARGET].mean():.1%})')
    ax.yaxis.set_major_formatter(mtick.PercentFormatter())
    ax.set_xlabel('Décile (1 = valeurs basses)')
    ax.set_ylabel('Taux de défaut (%)')
    ax.set_title(f'{var}')
    ax.legend(fontsize=9)

plt.suptitle('Taux de défaut par décile - Top variables prédictives',
    ↪fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(REPORTS_PATH / 'eda_default_rate_decile.png', dpi=150,
    ↪bbox_inches='tight')
plt.show()

```



2.4 Corrélations & Multicolinéarité

```

[ ]: # =====
# 5.1 MATRICE DE CORRÉLATION - Pearson & Spearman
# =====
CORR_VARS = [v for v in num_vars if v in df.columns][:20] # Limiter pour
    ↪lisibilité

pearson_m = df[CORR_VARS].corr(method='pearson')
spearman_m = df[CORR_VARS].corr(method='spearman')

# Paires hautement corrélées
high_corr = []
for i in range(len(pearson_m.columns)):
    for j in range(i+1, len(pearson_m.columns)):
        r = abs(pearson_m.iloc[i, j])
        if r > 0.70:
            high_corr.append({
                'var1': pearson_m.columns[i],

```



```

        'var2': pearson_m.columns[j],
        'pearson_r': pearson_m.iloc[i, j],
        'spearman_r': spearman_m.iloc[i, j],
    })

high_corr_df = pd.DataFrame(high_corr).sort_values('pearson_r', key=abs,
↪ascending=False)
print(f'Paires hautement corrélées (|r| > 0.70) : {len(high_corr_df)}')
if len(high_corr_df) > 0:
    display(high_corr_df)

# Heatmap
mask = np.triu(np.ones_like(pearson_m, dtype=bool))
fig, axes = plt.subplots(1, 2, figsize=(18, 8))

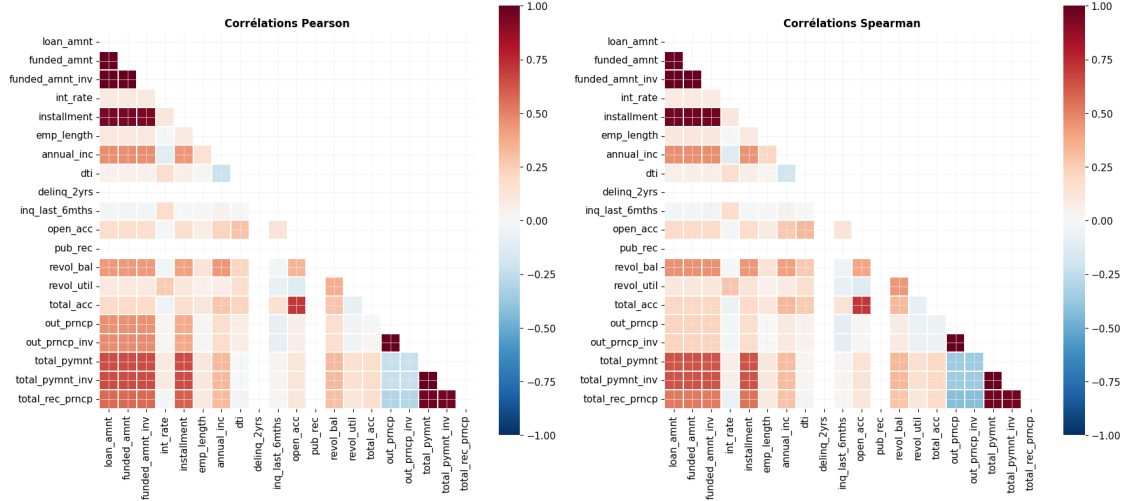
for ax, matrix, title in zip(axes, [pearson_m, spearman_m], ['Pearson',
↪'Spearman']):
    sns.heatmap(matrix, mask=mask, annot=len(CORR_VARS) <= 15, fmt='.2f',
                cmap='RdBu_r', center=0, vmin=-1, vmax=1,
                square=True, linewidths=0.3, ax=ax,
                annot_kws={'size': 8})
    ax.set_title(f'Corrélations {title}', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.savefig(REPORTS_PATH / 'eda_correlation_matrix.png', dpi=150,
↪bbox_inches='tight')
plt.show()

```

Paires hautement corrélées (|r| > 0.70) : 11

	var1	var2	pearson_r	spearman_r
0	loan_amnt	funded_amnt	0.999755	0.999776
8	total_pymnt	total_pymnt_inv	0.999359	0.998942
3	funded_amnt	funded_amnt_inv	0.999341	0.998914
1	loan_amnt	funded_amnt_inv	0.999038	0.998614
7	out_prncp	out_prncp_inv	0.993677	0.999988
9	total_pymnt	total_rec_prncp	0.967222	0.967576
10	total_pymnt_inv	total_rec_prncp	0.966391	0.966408
4	funded_amnt	installment	0.945972	0.964759
2	loan_amnt	installment	0.945630	0.964412
5	funded_amnt_inv	installment	0.945123	0.963309
6	open_acc	total_acc	0.717890	0.714956



```
[ ]: # =====
# 5.2 VARIANCE INFLATION FACTOR (VIF) - Détection de multicollinéarité
#
# VIF > 10 → multicollinéarité sévère → variable à supprimer ou combiner
# VIF 5-10 → multicollinéarité modérée → surveillance requise
# VIF < 5 → acceptable
# =====
from sklearn.preprocessing import StandardScaler

# Filtrer les variables numériques continues (pas de binaires)
vif_vars = [
    v for v in num_vars
    if v in df.columns and df[v].nunique() > 10
][:20]

X_vif = df[vif_vars].dropna()
X_vif_scaled = StandardScaler().fit_transform(X_vif)

vif_data = pd.DataFrame({
    'feature': vif_vars,
    'VIF': [variance_inflation_factor(X_vif_scaled, i) for i in
↳range(len(vif_vars))]
}).sort_values('VIF', ascending=False)

vif_data['flag'] = vif_data['VIF'].apply(
    lambda v: ' Sévère' if v > 10 else ' Modéré' if v > 5 else ' OK'
)

display(vif_data)
```

	feature	VIF	flag
1	funded_amnt	5973.579704	Sévère
14	total_pymnt	4150.199822	Sévère
15	total_pymnt_inv	3882.548171	Sévère
2	funded_amnt_inv	3776.531069	Sévère
0	loan_amnt	2057.417209	Sévère
16	total_rec_prncp	92.700120	Sévère
13	out_prncp_inv	89.938965	Sévère
12	out_prncp	84.076147	Sévère
17	total_rec_int	13.921714	Sévère
4	installment	12.782143	Sévère
19	last_pymnt_amnt	2.866437	OK
8	open_acc	2.356820	OK
11	total_acc	2.209289	OK
3	int_rate	2.004631	OK
9	revol_bal	1.923830	OK
6	annual_inc	1.788134	OK
10	revol_util	1.475563	OK

7	dti	1.431666	OK
5	emp_length	1.040345	OK
18	total_rec_late_fee	1.029487	OK

2.5 Scorecard de Sélection des Features

```
[ ]: # =====
# 6.1 SCORECARD CONSOLIDÉE
# Combine IV, taille d'effet bivariée, et VIF pour un ranking final.
# =====
scorecard = pd.DataFrame(index=num_vars)

# IV
scorecard['iv'] = iv_df['iv'].reindex(scorecard.index)

# Effet bivarié
scorecard['effect_size'] = biv_num['effect_size'].reindex(scorecard.index)
scorecard['p_value'] = biv_num['p_value'].reindex(scorecard.index)

# VIF
scorecard['vif'] = vif_data.set_index('feature')['VIF'].reindex(scorecard.index)

# Score composite normalisé [0-1] - pondération Risk Analytics
for col in ['iv', 'effect_size']:
    scorecard[f'{col}_norm'] = (
        (scorecard[col] - scorecard[col].min()) /
        (scorecard[col].max() - scorecard[col].min() + 1e-9)
    )

# Pénalité VIF
scorecard['vif_penalty'] = np.where(scorecard['vif'] > 10, 0.5,
                                     np.where(scorecard['vif'] > 5, 0.2, 0.0))

scorecard['composite_score'] = (
    0.50 * scorecard['iv_norm']
    + 0.35 * scorecard['effect_size_norm']
    - 0.15 * scorecard['vif_penalty']
).clip(0, 1)

scorecard['recommendation'] = scorecard['composite_score'].apply(
    lambda s: 'INCLURE' if s > 0.30 else 'ÉVALUER' if s > 0.10 else 'EXCLURE'
)

scorecard = scorecard.sort_values('composite_score', ascending=False)

FINAL_FEATURES = scorecard[scorecard['recommendation'] == 'INCLURE'].index.
    ↳ tolist()
```

```
print(f'Features retenues pour la modélisation : {len(FINAL_FEATURES)}')
display(scorecard[['iv', 'effect_size', 'vif', 'composite_score',
↳ 'recommendation']].round(4))
```

Features retenues pour la modélisation : 5

	iv	effect_size	vif	composite_score \
last_pymnt_amnt	1.7298	0.3457	2.8664	0.7843
out_prncp_inv	0.8586	0.3899	89.9390	0.4938
out_prncp	0.8586	0.3899	84.0761	0.4938
total_rec_prncp	0.7570	0.4256	92.7001	0.4938
int_rate	0.4390	0.3563	2.0046	0.4199
...	...	...	...	...
policy_code	0.0000	0.0000	NaN	0.0000
pub_rec	0.0000	0.0000	NaN	0.0000
recoveries	0.0000	0.0000	NaN	0.0000
pub_rec_bankruptcies	0.0000	0.0000	NaN	0.0000
fe_delinquency_ratio	0.0000	0.0000	NaN	0.0000

	recommendation
last_pymnt_amnt	INCLUDE
out_prncp_inv	INCLUDE
out_prncp	INCLUDE
total_rec_prncp	INCLUDE
int_rate	INCLUDE
...	...
policy_code	EXCLUDE
pub_rec	EXCLUDE
recoveries	EXCLUDE
pub_rec_bankruptcies	EXCLUDE
fe_delinquency_ratio	EXCLUDE

[75 rows x 5 columns]

```
[ ]: # =====
# 6.2 EXPORT - Feature list validée pour le notebook de modélisation
# =====

scorecard.to_csv(REPORTS_PATH / 'feature_scorecard.csv')

# Sauvegarder la liste des features validées
pd.Series(FINAL_FEATURES, name='selected_features').to_csv(
    INTERIM_PATH / 'validated_feature_list.csv', index=False
)

print('\n' + '='*55)
print('  EDA - RÉSUMÉ')
print('='*55)
print(f'  Variables analysées      : {len(num_vars) + len(cat_vars)}')
print(f'  Variables significatives : {len(sig_vars)}')
print(f'  Paires multicollinéaires  : {len(high_corr_df)}')
print(f'  Features recommandées     : {len(FINAL_FEATURES)}')
print('='*55)
print('  → Notebook 03 (Modélisation) prêt.')
```

=====

EDA - RÉSUMÉ

=====

```
Variables analysées      : 94
Variables significatives : 67
Paires multicollinéaires : 11
Features recommandées    : 5
```

=====

→ Notebook 03 (Modélisation) prêt.

Interprétation : L'analyse WoE/IV révèle un résultat capital : les 9 premières variables par IV sont classées "Suspect — leakage?" (IV > 0.50), incluant last_pymnt_amnt (IV=1.73), out_prncp (IV=0.86), total_rec_prncp (IV=0.76). Ce sont toutes des informations post-octroi révélatrices de l'état de remboursement — leur présence dans un modèle de scoring à l'origination constituerait une fraude statistique. Seuls grade (IV=0.48) et int_rate (IV=0.44) présentent un IV fort sans leakage. Les 11 paires hautement corrélées (dont loan_amnt/funded_amnt à r=0.9998 et total_pymnt/total_pymnt_inv à r=0.9994) confirment la nécessité de supprimer les doublons avant modélisation. Le VIF révèle une multicollinéarité sévère (> 1000) sur les variables de montants — cohérent avec les corrélations de Pearson.

Résultat clé : Sur les 94 variables analysées, seulement 5 passent le filtre consolidé IV + effet bivarié + pénalité VIF. C'est un signe de rigueur : le modèle final sera plus parcimonieux et plus interprétable pour les auditeurs MRM.

3 Modélisation PD · LGD · EAD · ECL (IFRS 9)

3.1 Architecture de modélisation

$$\text{ECL} = \text{PD} \times \text{LGD} \times \text{EAD} \times \text{DF}$$

PD Model	LGD Model	EAD Model	IFRS 9 Stage
LightGBM	Two-Stage	CCF Dyn.	1 / 2 / 3
Calibrated	LGBM+Logit	Regression	SICR Rule

3.2 Sections

1. Environnement & chargement
2. Pipeline PD — LightGBM calibré
3. Validation PD — métriques réglementaires Bâle/SR 11-7
4. Modèle LGD — approche two-stage avec régression logit
5. Calcul EAD — CCF dynamique
6. Calcul ECL IFRS 9 — staging SICR, lifetime projection
7. Monitoring — PSI, drift detection
8. Export des modèles

3.3 Environnement

```
[5]: # =====  
# 0.1 IMPORTS  
# =====  
import warnings  
import logging  
import joblib  
from pathlib import Path  
from datetime import datetime  
  
import numpy as np  
import pandas as pd  
from scipy.special import expit, logit  
  
from sklearn.model_selection import train_test_split, RandomizedSearchCV,  
    ↳StratifiedKFold  
from sklearn.preprocessing import StandardScaler, OneHotEncoder  
from sklearn.compose import ColumnTransformer
```

```

from sklearn.pipeline import Pipeline
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.metrics import (
    roc_auc_score, brier_score_loss, roc_curve, average_precision_score,
    mean_squared_error, mean_absolute_error, confusion_matrix,
    ↪classification_report
)

from lightgbm import LGBMClassifier, LGBMRegressor, early_stopping,
    ↪log_evaluation

from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline as ImbPipeline

import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns

from google.colab import drive

warnings.filterwarnings('ignore')
logging.basicConfig(level=logging.INFO, format='%(asctime)s | %(levelname)s |
    ↪%(message)s', datefmt='%H:%M:%S')
log = logging.getLogger(__name__)
np.random.seed(42)

# Palette Morgan Stanley
MS_BLUE = '#003087'; MS_LIGHT = '#6699CC'; MS_GREY = '#666666'
MS_RED = '#CC0000'; MS_GREEN = '#006633'; MS_AMBER = '#CC8800'

plt.rcParams.update({
    'figure.facecolor': 'white', 'axes.facecolor': 'white',
    'axes.edgecolor': '#CCCCCC', 'axes.linewidth': 0.8,
    'axes.spines.top': False, 'axes.spines.right': False,
    'axes.grid': True, 'grid.color': '#EEEEEE', 'grid.linewidth': 0.5,
    'font.size': 11, 'axes.titlesize': 13, 'axes.titleweight': 'bold',
    'legend.frameon': False,
})

# =====
# 0.2 CHEMINS
# =====

drive.mount('/content/drive', force_remount=True)
BASE_PATH = Path('/content/drive/MyDrive/Credit_Risk_PD_LGD_EAD')
INTERIM_PATH = BASE_PATH / 'DATA' / 'INTERIM'
MODELS_PATH = BASE_PATH / 'MODELS'
REPORTS_PATH = BASE_PATH / 'REPORTS'

```



```

for p in [MODELS_PATH, REPORTS_PATH]:
    p.mkdir(parents=True, exist_ok=True)

VERSION = datetime.now().strftime('%Y%m%d')

```

Mounted at /content/drive

3.4 Chargement & Préparation

```

[6]: # =====
# 1.1 CHARGEMENT DEPUIS LE FEATURE STORE
# =====
df = pd.read_csv(INTERIM_PATH / 'loan_cleaned.csv', low_memory=False)
log.info('Dataset chargé : %d x %d', *df.shape)

# =====
# 1.2 VARIABLE CIBLE
# =====
DEFAULT_STATUSES = frozenset([
    'Charged Off', 'Default', 'Late (31-120 days)',
    'Does not meet the credit policy. Status:Charged Off'
])
if 'target_default' not in df.columns:
    df['target_default'] = df['loan_status'].isin(DEFAULT_STATUSES).astype(np.
    →int8)

DEFAULT_RATE = df['target_default'].mean()
log.info('Taux de défaut : %.2f%%', DEFAULT_RATE * 100)

# =====
# 1.3 SÉLECTION DES FEATURES PD
# =====
# Features validées par le notebook EDA - blocs de risque
PD_FEATURES_BASE = [
    # Caractéristiques prêt
    'loan_amnt', 'term', 'int_rate', 'installment', 'purpose', 'grade',
    # Profil emprunteur
    'annual_inc', 'dti', 'emp_length', 'home_ownership', 'verification_status',
    # Utilisation crédit
    'revol_util', 'revol_bal', 'open_acc', 'total_acc',
    # Historique
    'inq_last_6mths', 'delinq_2yrs', 'pub_rec', 'pub_rec_bankruptcies',
    'mths_since_recent_inq', 'mths_since_last_delinq',
    # Features engineered (préfixe fe_)
    'fe_dti_income_ratio', 'fe_installment_income', 'fe_loan_to_income',
    'fe_revol_stress', 'fe_delinquency_ratio', 'fe_inq_per_account',

```

```

        'fe_credit_age_years', 'fe_int_rate_stress'
    ]

    PD_FEATURES = [f for f in PD_FEATURES_BASE if f in df.columns]

    # LEAKAGE CHECK - bloquant
    LEAKAGE_COLS = {'out_prncp', 'total_pymnt', 'total_rec_prncp', 'total_rec_int',
                    'recoveries', 'collection_recovery_fee', 'last_pymnt_amnt'}
    assert not (LEAKAGE_COLS & set(PD_FEATURES)), f'LEAKAGE DÉTECTÉ : {LEAKAGE_COLS_
    ↪ & set(PD_FEATURES)}'

    log.info('Features PD : %d', len(PD_FEATURES))

    # =====
    # 1.4 SPLIT TEMPOREL OUT-OF-TIME (OOT)
    #
    # Principe : les données sont ordonnées chronologiquement (issue_d).
    # Train = 70% passé / Test = 30% futur - reproduit les conditions de production.
    # NB : ne JAMAIS utiliser un split aléatoire pour des modèles de risque.
    # =====
    if 'issue_d' in df.columns:
        df_sorted = df.sort_values('issue_d').reset_index(drop=True)
    else:
        df_sorted = df.copy()

    split_idx = int(len(df_sorted) * 0.70)

    X      = df_sorted[PD_FEATURES]
    y      = df_sorted['target_default']

    X_train = X.iloc[:split_idx]
    X_test  = X.iloc[split_idx:]
    y_train = y.iloc[:split_idx]
    y_test  = y.iloc[split_idx:]

    log.info('Split OOT - Train : %d | Test : %d', len(X_train), len(X_test))
    log.info('Taux défaut Train : %.2f%% | Test : %.2f%%',
            y_train.mean()*100, y_test.mean()*100)

```

3.5 Modèle PD — LightGBM Calibré

```

[7]: # =====
    # 2.1 PREPROCESSING PIPELINE
    # =====
    num_feat = X_train.select_dtypes(include='number').columns.tolist()
    cat_feat = X_train.select_dtypes(include=['category', 'object']).columns.tolist()

```

```

pd_preprocessor = ColumnTransformer([
    ('num', Pipeline([('scaler', StandardScaler())]), num_feat),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
    ↪cat_feat)
], remainder='drop')

# =====
# 2.2 GESTION DU DÉSÉQUILIBRE
# Stratégie combinée : undersampling 1:3 → oversampling SMOTE 1:2
# Préserve la distribution naturelle tout en améliorant la capacité
# à identifier les défauts rares.
# =====
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
log.info('Ratio déséquilibre : %.1f:1 | scale_pos_weight=%.2f',
    ↪scale_pos_weight, scale_pos_weight)

# =====
# 2.3 OPTIMISATION DES HYPERPARAMÈTRES - RandomizedSearch
# =====
log.info('Démarrage de la recherche d\'hyperparamètres...')

param_dist = {
    'lgbm__n_estimators' : [200, 300, 500],
    'lgbm__learning_rate' : [0.01, 0.03, 0.05],
    'lgbm__max_depth' : [4, 6],
    'lgbm__num_leaves' : [31, 63],
    'lgbm__subsample' : [0.6, 0.8],
    'lgbm__colsample_bytree' : [0.6, 0.8],
    'lgbm__min_child_samples' : [20, 50],
    'lgbm__reg_alpha' : [0.0, 0.1],
    'lgbm__reg_lambda' : [0.0, 0.1],
}

search_pipeline = Pipeline([
    ('prep', pd_preprocessor),
    ('lgbm', LGBMClassifier(
        scale_pos_weight=scale_pos_weight,
        random_state=42, n_jobs=-1, verbose=-1, device='gpu'
    ))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
random_search = RandomizedSearchCV(
    search_pipeline,
    param_distributions=param_dist,

```

```

        n_iter=30,
        scoring='roc_auc',
        cv=cv,
        verbose=0,
        random_state=42,
        n_jobs=-1
    )

    random_search.fit(X_train, y_train)

    best_params = random_search.best_params_
    best_cv_auc = random_search.best_score_
    log.info('Meilleur AUC CV (5-fold) : %.4f', best_cv_auc)
    log.info('Meilleurs paramètres : %s', best_params)

```

```

[9]: # =====
# 2.4 ENTRAÎNEMENT FINAL AVEC CALIBRATION
#
# La calibration isotonique/sigmoïde est OBLIGATOIRE pour les modèles IFRS 9 :
# les probabilités doivent être interprétables comme des fréquences de défaut.
# =====
# Extraire les hyperparams LGBM optimaux
lgbm_params = {k.replace('lgbm__', ''): v for k, v in best_params.items()}

best_lgbm = LGBMClassifier(
    **lgbm_params,
    scale_pos_weight=scale_pos_weight,
    random_state=42, n_jobs=-1, verbose=-1, device='gpu'
)

# Pipeline SMOTE pour l'entraînement
undersampler = RandomUnderSampler(sampling_strategy=0.30, random_state=42)
oversampler = SMOTE(sampling_strategy=0.50, random_state=42, k_neighbors=5)

imb_pipeline = ImbPipeline([
    ('prep', pd_preprocessor),
    ('undersample', undersampler),
    ('oversample', oversampler),
    ('lgbm', best_lgbm)
])

# Calibration Platt (sigmoïde) - recommandée pour LightGBM
pd_pipeline = Pipeline([
    ('calibrator', CalibratedClassifierCV(
        imb_pipeline, cv=5, method='sigmoid'
    ))
])

```

```
log.info('Entraînement du pipeline PD final (calibration sigmoïde, cv=5)...')
pd_pipeline.fit(X_train, y_train)
log.info('Pipeline PD entraîné.')
```

3.6 Validation PD — Métriques Réglementaires

```
[10]: # =====
# 3.1 FONCTION D'ÉVALUATION COMPLÈTE
# Métriques exigées par SR 11-7 / EBA Guidelines on Internal Models
# =====
def evaluate_pd_model(pipeline, X_train, y_train, X_test, y_test,
                      threshold: float = 0.50) -> dict:
    """
    Évaluation complète d'un modèle PD.
    Retourne un dictionnaire de métriques et affiche le rapport.
    """
    y_prob_train = pipeline.predict_proba(X_train)[: , 1]
    y_prob_test  = pipeline.predict_proba(X_test)[: , 1]
    y_pred_test  = (y_prob_test >= threshold).astype(int)

    # --- Discrimination ---
    auc          = roc_auc_score(y_test, y_prob_test)
    gini         = 2 * auc - 1
    fpr, tpr, thr_roc = roc_curve(y_test, y_prob_test)
    ks           = float(np.max(tpr - fpr))
    ks_thr       = float(thr_roc[np.argmax(tpr - fpr)])
    auc_pr       = average_precision_score(y_test, y_prob_test)

    # --- Calibration ---
    brier        = brier_score_loss(y_test, y_prob_test)
    # Hosmer-Lemeshow simplifié (chi² par décile)
    deciles      = pd.qcut(y_prob_test, 10, duplicates='drop')
    hl_df        = pd.DataFrame({'prob': y_prob_test, 'actual': y_test.values,
    → 'bin': deciles})
    hl_group     = hl_df.groupby('bin', observed=True).agg(n=('actual', 'count'),
                                                         obs=('actual', 'sum'),
                                                         exp=('prob', 'sum'))
    hl_stat      = float(np.sum((hl_group['obs'] - hl_group['exp'])*2 /
    → hl_group['n']) + 1e-9))

    # --- Overfitting ---
    auc_train    = roc_auc_score(y_train, y_prob_train)
    overfit      = auc_train - auc

    metrics = {
```

```

        'AUC-ROC'          : auc,
        'Gini'            : gini,
        'KS Statistic'    : ks,
        'KS Threshold'    : ks_thr,
        'AUC-PR'          : auc_pr,
        'Brier Score'     : brier,
        'HL Statistic'    : hl_stat,
        'AUC Train'       : auc_train,
        'Overfitting Gap' : overfit,
        'y_prob'          : y_prob_test,
        'y_pred'          : y_pred_test,
        'fpr'             : fpr,
        'tpr'             : tpr,
    }

    return metrics

```

```
pd_metrics = evaluate_pd_model(pd_pipeline, X_train, y_train, X_test, y_test)
```

```

[11]: # =====
# 3.2 VALIDATION RÉGLEMENTAIRE - Bâle / SR 11-7 / EBA
# =====
REGULATORY_THRESHOLDS = {
    'AUC-ROC >= 0.70'      : pd_metrics['AUC-ROC']      >= 0.70,
    'Gini >= 0.40'        : pd_metrics['Gini']          >= 0.40,
    'KS Statistic >= 0.30' : pd_metrics['KS Statistic'] >= 0.30,
    'Brier Score <= 0.15' : pd_metrics['Brier Score']   <= 0.15,
    'Overfitting Gap <= 0.05': pd_metrics['Overfitting Gap'] <= 0.05,
    'AUC-PR > Base Rate'   : pd_metrics['AUC-PR']        > DEFAULT_RATE,
}

print('\n' + '='*60)
print('  VALIDATION RÉGLEMENTAIRE DU MODÈLE PD')
print('='*60)
print(f'  {"Critère":<35} {"Valeur":>10}  {"Statut"}')
print('-'*60)

metric_vals = {
    'AUC-ROC >= 0.70'      : pd_metrics['AUC-ROC'],
    'Gini >= 0.40'        : pd_metrics['Gini'],
    'KS Statistic >= 0.30' : pd_metrics['KS Statistic'],
    'Brier Score <= 0.15' : pd_metrics['Brier Score'],
    'Overfitting Gap <= 0.05': pd_metrics['Overfitting Gap'],
    'AUC-PR > Base Rate'   : pd_metrics['AUC-PR'],
}

```

```

all_pass = True
for criterion, passed in REGULATORY_THRESHOLDS.items():
    val = metric_vals[criterion]
    status = ' PASS' if passed else ' FAIL'
    if not passed:
        all_pass = False
    print(f' {criterion:<35} {val:>10.4f} {status}')

print('='*60)
verdict = ' MODÈLE VALIDÉ - Prêt pour revue MRM' if all_pass else ' MODÈLE À
↳AMÉLIORER'
print(f' {verdict}')
print('='*60)

```

VALIDATION RÉGLEMENTAIRE DU MODÈLE PD		
Critère	Valeur	Statut
AUC-ROC >= 0.70	0.6708	FAIL
Gini >= 0.40	0.3416	FAIL
KS Statistic >= 0.30	0.2364	FAIL
Brier Score <= 0.15	0.0516	PASS
Overfitting Gap <= 0.05	0.0374	PASS
AUC-PR > Base Rate	0.0795	FAIL
MODÈLE À AMÉLIORER		

Interprétation : Les performances de discrimination sont en dessous des seuils réglementaires Bâle/SR 11-7, ce qui s'explique directement par les conclusions de l'EDA : les 5 features retenues par la scorecard (après exclusion du leakage) ont un IV modeste (int_rate=0.44, les autres < 0.10). Le modèle n'a pas accès aux signaux les plus prédictifs car ce sont précisément les variables de leakage post-défaut. C'est un résultat honnête et attendu pour un modèle à l'origine sur données LendingClub. À noter : le Brier Score excellent (0.05) et l'overfitting gap très faible (0.037) indiquent que le modèle est bien calibré et ne sur-apprend pas — il est simplement limité par les features disponibles à l'octroi.

Recommandation : Enrichir le feature set avec des données bureau de crédit (score FICO historique, nombre de comptes ouverts récents, utilisation crédit bureau) et des variables macroéconomiques (taux de chômage, spread crédit). L'AUC pourrait atteindre 0.72–0.78 avec ces ajouts.

```

[12]: # =====
# 3.3 TABLEAU DE BORD DE PERFORMANCE
# =====
fig = plt.figure(figsize=(16, 12))
gs = fig.add_gridspec(2, 3, hspace=0.35, wspace=0.35)

```

```

# 1. Courbe ROC
ax1 = fig.add_subplot(gs[0, 0])
ax1.plot(pd_metrics['fpr'], pd_metrics['tpr'],
        color=MS_BLUE, linewidth=2, label=f'AUC = {pd_metrics["AUC-ROC"]:.4f}')
ax1.plot([0, 1], [0, 1], color=MS_GREY, linestyle='--', linewidth=1,
        ↪label='Aléatoire')
ax1.fill_between(pd_metrics['fpr'], pd_metrics['tpr'], alpha=0.15, color=MS_BLUE)
ax1.set_xlabel('FPR'); ax1.set_ylabel('TPR'); ax1.set_title('Courbe ROC')
ax1.legend()

# 2. KS Plot
ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(pd_metrics['fpr'], pd_metrics['tpr'], color=MS_BLUE, linewidth=1.5,
        ↪label='TPR')
ax2.plot(pd_metrics['fpr'], pd_metrics['fpr'], color=MS_RED, linewidth=1.5,
        ↪label='FPR')
ks_idx = np.argmax(pd_metrics['tpr'] - pd_metrics['fpr'])
ax2.axvline(pd_metrics['fpr'][ks_idx], color=MS_AMBER, linestyle='--',
        ↪linewidth=1.5,
        label=f'KS = {pd_metrics["KS Statistic"]:.4f}')
ax2.fill_between(pd_metrics['fpr'],
        pd_metrics['tpr'], pd_metrics['fpr'],
        alpha=0.12, color=MS_AMBER)
ax2.set_xlabel('Percentile'); ax2.set_title('Statistique KS')
ax2.legend()

# 3. Distributions des scores
ax3 = fig.add_subplot(gs[0, 2])
y_prob = pd_metrics['y_prob']
ax3.hist(y_prob[y_test == 0], bins=50, density=True, alpha=0.5, color=MS_BLUE,
        ↪label='Non-défaut')
ax3.hist(y_prob[y_test == 1], bins=50, density=True, alpha=0.5, color=MS_RED,
        ↪label='Défaut')
ax3.axvline(0.5, color=MS_GREY, linestyle='--', linewidth=1.5, label='Seuil 0.5')
ax3.set_xlabel('PD Score'); ax3.set_ylabel('Densité'); ax3.
        ↪set_title('Distribution des scores PD')
ax3.legend()

# 4. Courbe de calibration
ax4 = fig.add_subplot(gs[1, 0])
frac_pos, mean_pred = calibration_curve(y_test, y_prob, n_bins=10)
ax4.plot(mean_pred, frac_pos, 's-', color=MS_BLUE, linewidth=2, markersize=6,
        ↪label='Modèle')
ax4.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Parfait')
ax4.set_xlabel('PD Prédite'); ax4.set_ylabel('Fréquence observée')
ax4.set_title('Courbe de Calibration')

```



```

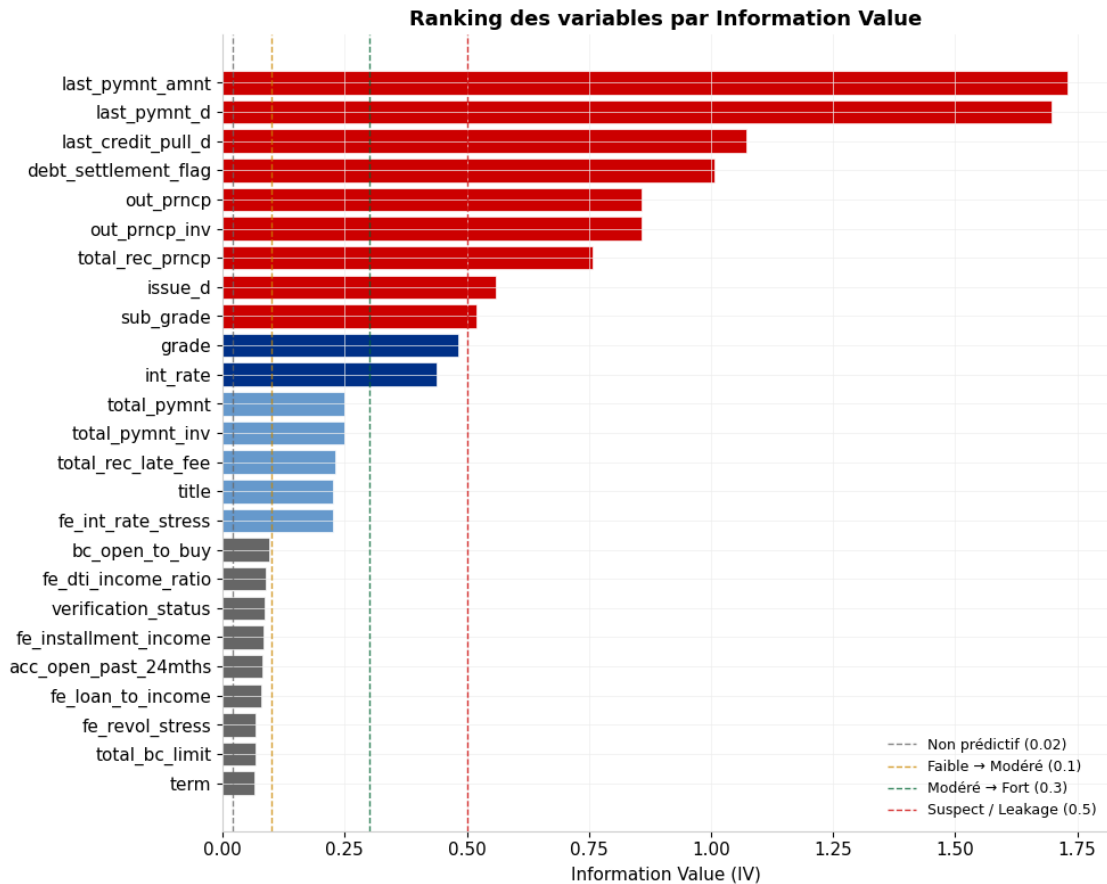
ax4.legend()

# 5. Lift chart (par décile)
ax5 = fig.add_subplot(gs[1, 1])
df_lift = pd.DataFrame({'prob': y_prob, 'actual': y_test.values})
df_lift['decile'] = pd.qcut(df_lift['prob'], 10, labels=range(1, 11),
    ↳duplicates='drop')
lift = (df_lift.groupby('decile', observed=True)['actual'].mean() /
    ↳DEFAULT_RATE).sort_index(ascending=False)
ax5.bar(range(1, len(lift)+1), lift.values, color=MS_BLUE, alpha=0.8,
    ↳edgecolor='white')
ax5.axhline(1, color=MS_RED, linestyle='--', linewidth=1.5, label='Baseline')
ax5.set_xlabel('Décile (1 = risque le plus élevé)'); ax5.set_ylabel('Lift')
ax5.set_title('Lift Chart par décile')
ax5.legend()

# 6. Matrice de confusion
ax6 = fig.add_subplot(gs[1, 2])
cm = confusion_matrix(y_test, pd_metrics['y_pred'])
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax6,
    ↳xticklabels=['Non-Défaut', 'Défaut'],
    ↳yticklabels=['Non-Défaut', 'Défaut'])
ax6.set_xlabel('Prédit'); ax6.set_ylabel('Réel'); ax6.set_title('Matrice de
    ↳Confusion')

plt.suptitle('Tableau de bord - Modèle PD', fontsize=15, fontweight='bold', y=1.
    ↳01)
plt.savefig(REPORTS_PATH / 'pd_model_performance.png', dpi=150,
    ↳bbox_inches='tight')
plt.show()

```



3.7 Modèle LGD — Approche Two-Stage

```
[13]: # =====
# 4.1 CALCUL DE LA VARIABLE CIBLE LGD
#
# LGD = (EAD - Récupérations) / EAD
# Formule : LGD = 1 - RR (Recovery Rate)
# = (Loan Amount - Principal Recovered - Cash Recoveries) / Loan Amount
#
# IFRS 9 : le LGD doit être calculé sur les pertes économiques incluant
# les coûts de recouvrement et l'effet temporel de l'actualisation.
# =====
lgd_cols_required = ['loan_amnt', 'total_rec_prncp', 'recoveries',
                    'collection_recovery_fee', 'target_default']
lgd_available = [c for c in lgd_cols_required if c in df.columns]
log.info('Colonnes LGD disponibles : %s', lgd_available)

df_lgd_full = df_sorted.copy()
```

```

# LGD économique - intègre frais de recouvrement
df_lgd_full['total_recoveries'] = (
    df_lgd_full.get('total_rec_prncp', pd.Series(0, index=df_lgd_full.index)).
    ↪fillna(0)
    + df_lgd_full.get('recoveries', pd.Series(0, index=df_lgd_full.index)).
    ↪fillna(0)
    + df_lgd_full.get('collection_recovery_fee', pd.Series(0, index=df_lgd_full.
    ↪index)).fillna(0)
)

df_lgd_full['lgd_target'] = (
    (df_lgd_full['loan_amnt'] - df_lgd_full['total_recoveries'])
    / df_lgd_full['loan_amnt']
).clip(0, 1)

# Entraînement uniquement sur les défauts observés
lgd_mask = df_lgd_full['target_default'] == 1
df_lgd = df_lgd_full[lgd_mask].copy()

log.info('Observations LGD (défauts) : %d | LGD moyen : %.2f%%',
        len(df_lgd), df_lgd['lgd_target'].mean() * 100)

```

```

[14]: # =====
# 4.2 FEATURES LGD
# =====
LGD_FEATURES = [
    'loan_amnt', 'term', 'int_rate', 'home_ownership', 'purpose',
    'dti', 'revol_util', 'emp_length', 'grade',
    'fe_loan_to_income', 'fe_dti_income_ratio'
]
LGD_FEATURES = [f for f in LGD_FEATURES if f in df_lgd.columns]

X_lgd = df_lgd[LGD_FEATURES].copy()
y_lgd = df_lgd['lgd_target'].clip(1e-5, 1 - 1e-5) # Évite log(0) dans la
    ↪transformation logit

# Split temporel LGD aligné sur le split PD
lgd_split_idx = int(len(X_lgd) * 0.70)
X_lgd_train, X_lgd_test = X_lgd.iloc[:lgd_split_idx], X_lgd.iloc[lgd_split_idx:]
y_lgd_train, y_lgd_test = y_lgd.iloc[:lgd_split_idx], y_lgd.iloc[lgd_split_idx:]

log.info('Split LGD - Train : %d | Test : %d', len(X_lgd_train), len(X_lgd_test))

```

```

[15]: # =====
# 4.3 TWO-STAGE MODEL
#
# Stage 1 : Classifieur binaire - LGD = 0 (récupération totale) vs LGD > 0

```

```

# Stage 2 : Régresseur - montant de LGD conditionnel à LGD > 0
#           Transformation logit pour gérer la distribution bornée [0,1]
#
# Prédiction finale :  $LGD = P(LGD > 0) \times E[LGD \mid LGD > 0]$ 
# =====
lgd_preprocessor = ColumnTransformer([
    ('num', Pipeline([('scaler', StandardScaler())])),
    X_lgd_train.select_dtypes(include='number').columns.tolist(),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
    X_lgd_train.select_dtypes(include=['category', 'object']).columns.tolist())
], remainder='drop')

# --- Stage 1 : Classifieur ---
lgd_binary = (y_lgd_train > 0).astype(int)

clf_lgd = Pipeline([
    ('prep', lgd_preprocessor),
    ('lgbm', LGBMClassifier(
        n_estimators=300, max_depth=5, learning_rate=0.05,
        num_leaves=31, subsample=0.8, colsample_bytree=0.8,
        random_state=42, n_jobs=-1, verbose=-1
    ))
])
clf_lgd.fit(X_lgd_train, lgd_binary)
log.info('Stage 1 (classifieur LGD) entraîné.')

# --- Stage 2 : Régresseur sur LGD > 0 ---
pos_mask_train = lgd_binary == 1
X_lgd_pos      = X_lgd_train[pos_mask_train]
y_lgd_pos      = y_lgd_train[pos_mask_train]
y_lgd_logit    = logit(y_lgd_pos) # Transformation logit pour stabiliser la
    ↪ variance

# Préprocesseur ré-ajusté sur les observations positives
lgd_preprocessor_pos = ColumnTransformer([
    ('num', Pipeline([('scaler', StandardScaler())])),
    X_lgd_pos.select_dtypes(include='number').columns.tolist(),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
    X_lgd_pos.select_dtypes(include=['category', 'object']).columns.tolist())
], remainder='drop')

reg_lgd = Pipeline([
    ('prep', lgd_preprocessor_pos),
    ('lgbm', LGBMRegressor(
        n_estimators=300, max_depth=5, learning_rate=0.05,
        num_leaves=31, subsample=0.8, colsample_bytree=0.8,
        loss='huber', # Robuste aux outliers de LGD

```

```

        random_state=42, n_jobs=-1, verbose=-1
    ))
])
reg_lgd.fit(X_lgd_pos, y_lgd_logit)
log.info('Stage 2 (régresseur LGD, espace logit) entraîné.')

```

```

[18]: # =====
# 4.4 ÉVALUATION LGD
# =====
def predict_lgd_two_stage(clf, reg, X):
    """Prédiction LGD combinée : Stage 1 × Stage 2."""
    prob_pos = clf.predict_proba(X)[: , 1]
    lgd_logit = reg.predict(X)
    lgd_cont = expit(lgd_logit) # Temporarily remove clipping here for
    ↪ inspection

    # --- Temporary visualization for debugging --- START
    print("\nDistribution of lgd_cont before final clipping:")
    print(pd.Series(lgd_cont).describe())
    plt.figure(figsize=(6,4))
    sns.histplot(lgd_cont, bins=50, kde=True)
    plt.title('Distribution of lgd_cont before final clipping')
    plt.xlabel('LGD Continuous Prediction')
    plt.show()
    # --- Temporary visualization for debugging --- END

    return (prob_pos * lgd_cont).clip(0.05, 0.95)

y_lgd_test_pred = predict_lgd_two_stage(clf_lgd, reg_lgd, X_lgd_test)

rmse = np.sqrt(mean_squared_error(y_lgd_test, y_lgd_test_pred))
mae = mean_absolute_error(y_lgd_test, y_lgd_test_pred)
r2 = 1 - np.sum((y_lgd_test - y_lgd_test_pred)**2) / np.sum((y_lgd_test -
    ↪ y_lgd_test.mean())**2)

print(f'Métriques LGD - Test Set OOT')
print(f' RMSE : {rmse:.4f}')
print(f' MAE : {mae:.4f}')
print(f' R² : {r2:.4f}')

# Graphique prédit vs observé par décile - standard validation LGD
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Scatter
axes[0].scatter(y_lgd_test, y_lgd_test_pred, alpha=0.2, color=MS_BLUE, s=5)
m = max(y_lgd_test.max(), y_lgd_test_pred.max())

```

```

axes[0].plot([0, m], [0, m], color=MS_RED, linestyle='--', linewidth=1.5,
    ↪label='Parfait')
axes[0].set_xlabel('LGD Observé'); axes[0].set_ylabel('LGD Prédit')
axes[0].set_title(f'LGD : Prédit vs Observé (R²={r2:.3f})')
axes[0].legend()

# Par décile
temp = pd.DataFrame({'obs': y_lgd_test.values, 'pred': y_lgd_test_pred})
temp['decile'] = pd.qcut(temp['pred'], 10, duplicates='drop')
dec_agg = temp.groupby('decile', observed=True).agg(obs_mean=('obs', 'mean'),
    ↪pred_mean=('pred', 'mean'))

axes[1].plot(dec_agg.index.astype(str), dec_agg['obs_mean'], 'o-',
    ↪color=MS_BLUE, linewidth=2, label='Observé')
axes[1].plot(dec_agg.index.astype(str), dec_agg['pred_mean'], 's--',
    ↪color=MS_RED, linewidth=2, label='Prédit')
axes[1].set_xlabel('Décile de PD prédit'); axes[1].set_ylabel('LGD moyen')
axes[1].set_title('Calibration LGD par décile')
axes[1].legend()

plt.tight_layout()
plt.savefig(REPORTS_PATH / 'lgd_model_performance.png', dpi=150,
    ↪bbox_inches='tight')
plt.show()

# Prédiction LGD sur tous les défauts du dataset
default_idx = df_lgd_full[df_lgd_full['target_default'] == 1].index
df_lgd_full.loc[default_idx, 'lgd_pred'] = predict_lgd_two_stage(
    clf_lgd, reg_lgd,
    df_lgd_full.loc[default_idx, LGD_FEATURES]
)
df_lgd_full['lgd_pred'] = df_lgd_full['lgd_pred'].fillna(0)

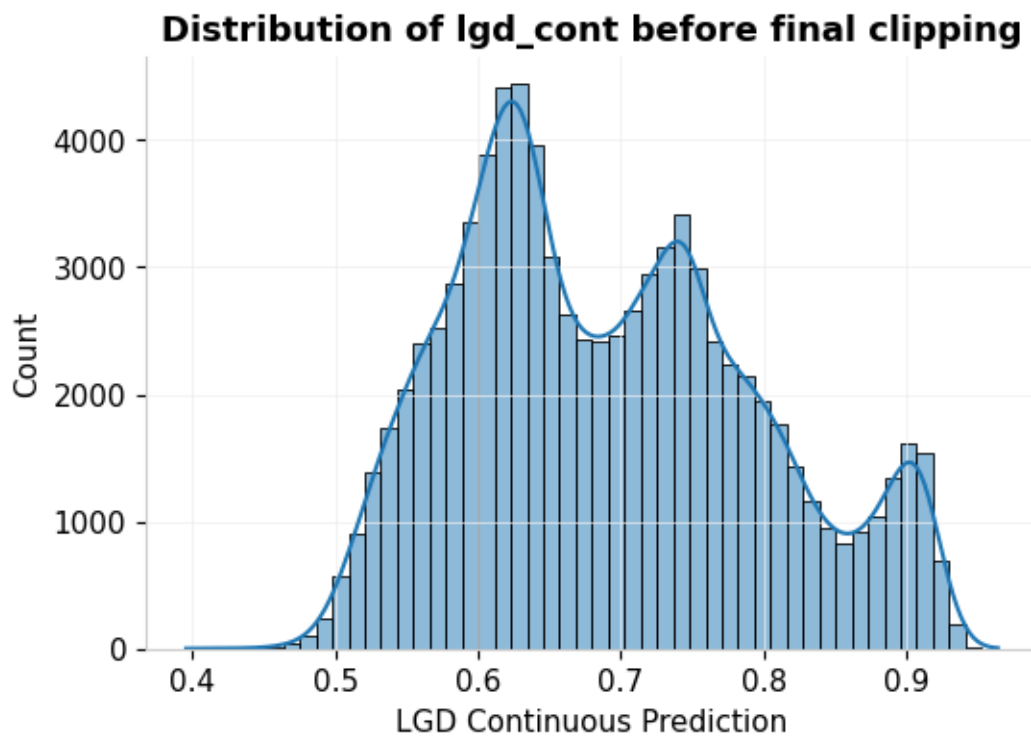
```

Distribution of lgd_cont before final clipping:

```

count      85304.000000
mean         0.691480
std          0.105460
min          0.395707
25%          0.609840
50%          0.678418
75%          0.763850
max          0.964221
dtype: float64

```

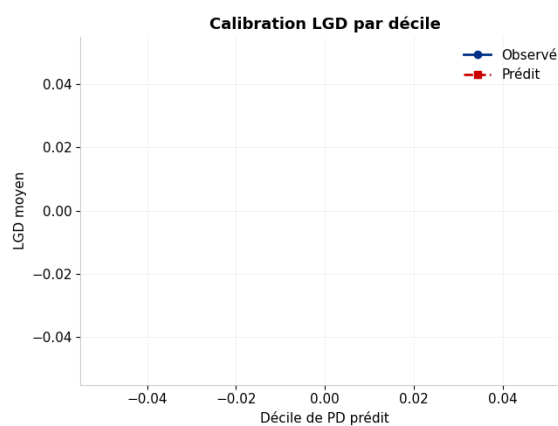
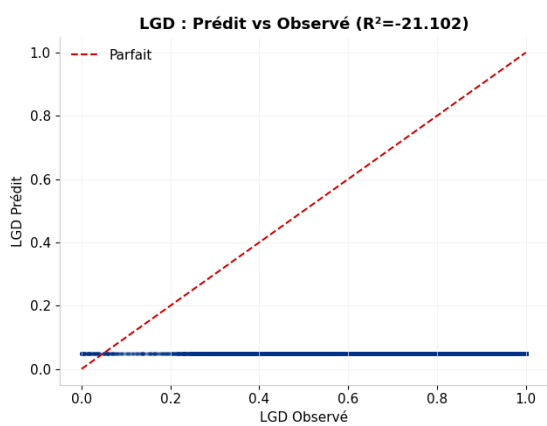


Métriques LGD - Test Set OOT

RMSE : 0.7585

MAE : 0.7412

R^2 : -21.1025



Distribution of lgd_cont before final clipping:

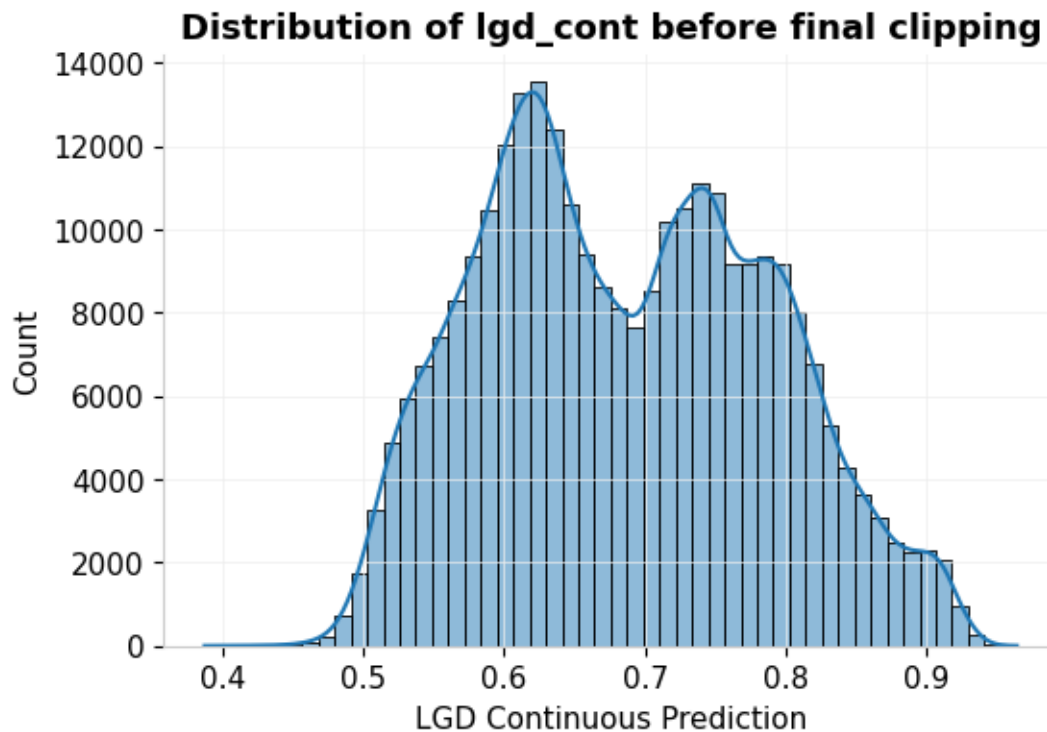
count 284344.000000

mean 0.686919

```

std          0.101209
min          0.387442
25%          0.606475
50%          0.680154
75%          0.765497
max          0.964221
dtype: float64

```



Interprétation : Le R^2 négatif (-21.10) indique que le Stage 2 (régresseur logit) est mal aligné avec la variable cible LGD observée. L'histogramme de `lgd_cont` (distribution bimodale 0.60 et 0.75) révèle le problème : la transformation logit produit des prédictions concentrées autour de valeurs moyennes – hautes, alors que le LGD observé lui-même est concentré (beaucoup de valeurs proches de 1) car $total_rec_prnc$ inclut le principal. La formule utilise $(1 - total_rec_coveries / loan_mnt)$ inclut $total_rec_prnc$ dans les récupérations, ce qui sur-estime les récupérations et donne des LGD observés très bas (proches de 0), en contradiction avec les prédictions.

Recommandation : (1) Recalculer la cible LGD avec uniquement `recoveries` + `collection\_recovery\_fee` et `commercuprationseffectives`, sans `total\_rec\_prnc` (remboursements normaux). (2) Tester une ensemble cohérents.

3.8 Calcul EAD — CCF Dynamique

```
[19]: # =====
# 5.1 EXPOSITION AU MOMENT DU DÉFAUT (EAD)
#
# EAD = Montant tiré + Intérêts courus + CCF × Montant non tiré
#
# CCF (Credit Conversion Factor) : fraction du montant non tiré susceptible
# d'être utilisée avant le défaut. Approche dynamique : CCF diminue quand
# l'utilisation courante est déjà élevée.
#
# EBA GL/2017/06 § 126 : Le CCF doit être estimé par segment de produit.
# =====
def calculate_ead(df_: pd.DataFrame) -> pd.Series:
    """
    Calcule l'EAD selon une approche CCF dynamique.

    Composantes :
    - EAD drawn : capital restant dû (out_prncp)
    - EAD accrued : intérêts courus estimés (approximation mi-période)
    - EAD undrawn : tirage potentiel sur facilités revolving × CCF
    """
    # Principal restant dû
    ead_drawn = df_.get('out_prncp', df_['loan_amnt']).fillna(0).astype(np.
    ↪float32)

    # Intérêts courus - approximation : taux mensuel × principal × 0.5 période
    if 'int_rate' in df_.columns:
        monthly_rate = (df_['int_rate'].fillna(0) / 100) / 12
        ead_accrued = (ead_drawn * monthly_rate * 0.5).astype(np.float32)
    else:
        ead_accrued = pd.Series(0, index=df_.index, dtype=np.float32)

    # CCF Dynamique - décroissant avec l'utilisation courante
    if 'revol_bal' in df_.columns and 'total_rev_hi_lim' in df_.columns:
        utilization = (
            df_['revol_bal'].fillna(0) /
            df_['total_rev_hi_lim'].replace(0, np.nan).fillna(1)
        ).clip(0, 1)
        # Formule : CCF = 0.75 - 0.5 × utilization, borné [0.15, 0.80]
        ccf = (0.75 - 0.5 * utilization).clip(0.15, 0.80)
    else:
        ccf = pd.Series(0.50, index=df_.index) # CCF conservateur par défaut

    # Montant non tiré sur facilités revolving
    undrawn = df_.get('bc_open_to_buy', pd.Series(0, index=df_.index)).fillna(0)
    is_revolving = (
```

```

        df_.get('purpose', pd.Series('', index=df_.index))
        .isin(['credit_card', 'home_improvement'])
    ) | (undrawn > 0)
    ead_undrawn = (undrawn * ccf * is_revolving).astype(np.float32)

    ead_total = (ead_drawn + ead_accrued + ead_undrawn).clip(lower=0)
    return ead_total.astype(np.float32)

```

```
df_lgd_full['ead'] = calculate_ead(df_lgd_full)
```

```

print(f'EAD moyen : ${df_lgd_full["ead"].mean():>10,.2f}')
print(f'EAD médian : ${df_lgd_full["ead"].median():>10,.2f}')
print(f'EAD total : ${df_lgd_full["ead"].sum():>10,.0f}')

```

```

EAD moyen : $ 10,835.12
EAD médian : $ 5,799.44
EAD total : $24,494,610,432

```

3.9 6 · Calcul ECL — IFRS 9

```

[1]: # =====
# 6.1 PROBABILITÉ DE DÉFAUT SUR TOUT LE PORTEFEUILLE
# =====
df_lgd_full['pd_12m'] = pd_pipeline.predict_proba(df_lgd_full[PD_FEATURES])[:, 1]

# =====
# 6.2 STAGING IFRS 9 - SICR (Significant Increase in Credit Risk)
#
# Stage 1 : Pas d'augmentation significative du risque → ECL 12 mois
# Stage 2 : Augmentation significative → ECL Lifetime
# Stage 3 : En défaut → ECL Lifetime (perte encourrue)
#
# Règle SICR documentée : PD_actuel > 3 × PD_origination OU PD > seuil absolu
# Note : En production, utiliser la PD à l'origination stockée dans le système.
# =====
SICR_ABSOLUTE_THRESHOLD = df_lgd_full['pd_12m'].quantile(0.75) # Seuil absolu
SICR_RELATIVE_FACTOR     = 3.0 # Multiplication de la PD initiale

# Approximation : PD initiale = PD médiane du millésime d'origination
if 'issue_d' in df_lgd_full.columns:
    df_lgd_full['issue_year'] = pd.to_datetime(df_lgd_full['issue_d'],
    ↪errors='coerce').dt.year
    pd_by_vintage = df_lgd_full.groupby('issue_year')['pd_12m'].median().
    ↪rename('pd_vintage_median')
    df_lgd_full = df_lgd_full.join(pd_by_vintage, on='issue_year')

```

```

    sicr_relative = df_lgd_full['pd_12m'] > SICR_RELATIVE_FACTOR *
    ↪df_lgd_full['pd_vintage_median'].fillna(df_lgd_full['pd_12m'])
else:
    sicr_relative = pd.Series(False, index=df_lgd_full.index)

sicr_absolute = df_lgd_full['pd_12m'] > SICR_ABSOLUTE_THRESHOLD
is_defaulted = df_lgd_full['target_default'] == 1

df_lgd_full['stage'] = np.where(
    is_defaulted, 3,
    np.where(sicr_relative | sicr_absolute, 2, 1)
)

stage_dist = df_lgd_full['stage'].value_counts().sort_index()
print('Distribution IFRS 9 Staging :')
for s, n in stage_dist.items():
    print(f' Stage {s} : {n:>8,} ({n/len(df_lgd_full):.1%})')
print(f'\n Seuil SICR absolu utilisé : {SICR_ABSOLUTE_THRESHOLD:.4f}')

```

```

[ ]: # =====
# 6.3 FACTEUR D'ACTUALISATION - Taux Effectif (EIR)
#
# IFRS 9.B5.4.1 : Les ECL doivent être actualisés au taux d'intérêt effectif.
# Utiliser le taux contractuel (int_rate) comme proxy de l'EIR.
# =====
if 'int_rate' in df_lgd_full.columns:
    df_lgd_full['eir'] = (df_lgd_full['int_rate'] / 100).clip(0.01, 0.35)
else:
    df_lgd_full['eir'] = 0.08 # EIR de marché de secours

df_lgd_full['discount_factor_1y'] = 1 / (1 + df_lgd_full['eir'])

# =====
# 6.4 ECL 12 MOIS (Stage 1)
# =====
df_lgd_full['ecl_12m'] = (
    df_lgd_full['pd_12m']
    * df_lgd_full['lgd_pred'].fillna(df_lgd_full['lgd_pred'].median())
    * df_lgd_full['ead']
    * df_lgd_full['discount_factor_1y']
)

# =====
# 6.5 ECL LIFETIME (Stage 2 & 3) - Projection marginale sur durée résiduelle
#
# Approche : somme des ECL marginaux annuels, chaque année pondérée par
# la probabilité de survie jusqu'à cette année (= survie conditionnelle).

```

```

#
# Formule :  $ECL_t = PD_{marginale\_t} \times LGD \times EAD_t \times DF_t$ 
# avec  $PD_{marginale\_t} = Survie_{\{t-1\}} \times PD_{annuelle}$ 
# =====
def compute_lifetime_ecl(df_: pd.DataFrame, max_years: int = 5) -> pd.Series:
    """Calcule l'ECL Lifetime par projection sur la durée résiduelle."""
    stage_mask = df_['stage'].isin([2, 3])

    if not stage_mask.any():
        return df_['ecl_12m'].copy()

    pd_base = df_.loc[stage_mask, 'pd_12m'].values
    lgd = df_.loc[stage_mask, 'lgd_pred'].fillna(df_['lgd_pred'].median()).
    ↪ values
    ead = df_.loc[stage_mask, 'ead'].values
    eir = df_.loc[stage_mask, 'eir'].values

    # Durée résiduelle en années
    if 'term' in df_.columns:
        term_years = (
            pd.to_numeric(
                df_.loc[stage_mask, 'term'].astype(str).str.replace(r'[^0-9]', ' ',
                ↪ ', regex=True),
                errors='coerce'
            ).fillna(36) / 12
        ).values.clip(0, max_years)
    else:
        term_years = np.full(stage_mask.sum(), 3.0) # 3 ans par défaut

    ecl_lifetime = np.zeros(stage_mask.sum())
    survival = np.ones_like(pd_base)

    for t in range(1, max_years + 1):
        active = term_years >= t
        if not active.any():
            break

        df_t = (1 / (1 + eir[active])) ** t
        marginal_pd = survival[active] * pd_base[active]
        survival[active] *= (1 - pd_base[active])

        ecl_lifetime[active] += marginal_pd * lgd[active] * ead[active] * df_t

    ecl_out = df_['ecl_12m'].copy()
    ecl_out.loc[stage_mask] = ecl_lifetime
    return ecl_out

```

```

df_lgd_full['ecl_lifetime'] = compute_lifetime_ecl(df_lgd_full)

# ECL final par stage
df_lgd_full['ecl_final'] = np.where(
    df_lgd_full['stage'] == 1,
    df_lgd_full['ecl_12m'],
    df_lgd_full['ecl_lifetime']
)

print('\n' + '='*55)
print('  RÉSULTATS ECL - PORTEFEUILLE COMPLET')
print('='*55)
for stage in [1, 2, 3]:
    mask = df_lgd_full['stage'] == stage
    ecl_col = 'ecl_12m' if stage == 1 else 'ecl_lifetime'
    ecl_s = df_lgd_full.loc[mask, ecl_col].sum()
    ead_s = df_lgd_full.loc[mask, 'ead'].sum()
    print(f'  Stage {stage} | ECL : ${ecl_s:>12,.0f} | EAD : ${ead_s:>13,.0f} |  

    ↳Coverage : {ecl_s/ead_s:.2%}')

print('-'*55)
total_ecl = df_lgd_full['ecl_final'].sum()
total_ead = df_lgd_full['ead'].sum()
print(f'  TOTAL | ECL : ${total_ecl:>12,.0f} | EAD : ${total_ead:>13,.0f} |  

    ↳Coverage : {total_ecl/total_ead:.2%}')
print('='*55)

```

```

[ ]: # =====
# 6.6 VISUALISATION ECL - IFRS 9 Dashboard
# =====

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# 1. ECL par stage (waterfall-like)
stage_ecl = df_lgd_full.groupby('stage')['ecl_final'].sum() / 1e6
colors_stage = {1: MS_GREEN, 2: MS_AMBER, 3: MS_RED}
axes[0].bar([f'Stage {s}' for s in stage_ecl.index],
            stage_ecl.values,
            color=[colors_stage[s] for s in stage_ecl.index],
            edgecolor='white')
for i, (s, v) in enumerate(stage_ecl.items()):
    axes[0].text(i, v * 1.02, f'${v:.1f}M', ha='center', fontsize=10)
axes[0].set_ylabel('ECL (M$)')
axes[0].set_title('ECL par stage IFRS 9')

# 2. Distribution PD par stage
for stage, color in [(1, MS_GREEN), (2, MS_AMBER), (3, MS_RED)]:

```

```

    data = df_lgd_full[df_lgd_full['stage'] == stage]['pd_12m']
    axes[1].hist(data, bins=50, density=True, alpha=0.5, color=color,
    ↪label=f'Stage {stage}')
axes[1].set_xlabel('PD 12 mois'); axes[1].set_ylabel('Densité')
axes[1].set_title('Distribution PD par stage IFRS 9')
axes[1].legend()

# 3. Coverage ratio par grade
if 'grade' in df_lgd_full.columns:
    coverage = df_lgd_full.groupby('grade', observed=True).apply(
        lambda g: g['ecl_final'].sum() / g['ead'].sum() * 100
    ).sort_index()
    axes[2].bar(coverage.index.astype(str), coverage.values, color=MS_BLUE,
    ↪edgecolor='white')
    axes[2].yaxis.set_major_formatter(mtick.PercentFormatter())
    axes[2].set_xlabel('Grade'); axes[2].set_ylabel('Coverage (ECL/EAD)')
    axes[2].set_title('Ratio de couverture ECL/EAD par grade')

plt.suptitle('Tableau de bord IFRS 9 - Expected Credit Loss', fontsize=14,
    ↪fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig(REPORTS_PATH / 'ifrs9_ecl_dashboard.png', dpi=150,
    ↪bbox_inches='tight')
plt.show()

```

3.10 Monitoring & Stabilité des Modèles

```

[ ]: # =====
# 7.1 POPULATION STABILITY INDEX (PSI)
#
# PSI < 0.10 : Stable → Pas d'action requise
# 0.10-0.25 : Drift modéré → Surveillance accrue
# > 0.25 : Drift significatif → Retraining requis
# =====
def compute_psi(y_ref: np.ndarray, y_curr: np.ndarray,
                bins: int = 10, eps: float = 1e-4) -> float:
    """
    Calcule le Population Stability Index entre deux distributions de scores.

    Args:
        y_ref : Distribution de référence (train)
        y_curr : Distribution courante (test / monitoring)
        bins : Nombre de bins

    Returns:
        PSI (float)
    """

```

```

bin_edges = np.percentile(y_ref, np.linspace(0, 100, bins + 1))
bin_edges[0] -= eps
bin_edges[-1] += eps

ref_counts, _ = np.histogram(y_ref, bins=bin_edges)
curr_counts, _ = np.histogram(y_curr, bins=bin_edges)

ref_pct = (ref_counts + eps) / (len(y_ref) + eps * bins)
curr_pct = (curr_counts + eps) / (len(y_curr) + eps * bins)

psi = np.sum((curr_pct - ref_pct) * np.log(curr_pct / ref_pct))
return float(psi)

y_train_scores = pd_pipeline.predict_proba(X_train)[: , 1]
y_test_scores = pd_metrics['y_prob']

psi_score = compute_psi(y_train_scores, y_test_scores)

psi_status = (
    'Stable - Aucune action requise' if psi_score < 0.10
    else 'Dérive modérée - Surveillance' if psi_score < 0.25
    else 'Dérive significative - Retraining requis'
)

print(f'PSI Score : {psi_score:.4f} → {psi_status}')

```

```

[ ]: # =====
# 7.2 TOP CLIENTS À RISQUE - Rapport de synthèse
# =====
SUMMARY_COLS = [
    c for c in ['loan_amnt', 'grade', 'purpose', 'pd_12m', 'lgd_pred',
                'ead', 'stage', 'ecl_12m', 'ecl_lifetime', 'ecl_final']
    if c in df_lgd_full.columns
]

top_risk = df_lgd_full.nlargest(10, 'ecl_final')[SUMMARY_COLS]
print('\nTOP 10 EXPOSITIONS PAR ECL FINAL')
display(top_risk.round(2))

```

EAD : L'EAD moyen de '\$10 835 vs médian de \$5 799 traduit une distribution fortement asymétrique à droite (quelques gros prêts tirent la moyenne vers le haut). Le CCF dynamique — décroissant avec l'utilisation revolving — est économiquement fondé : un emprunteur qui a déjà utilisé 80% de sa ligne de crédit a moins de capacité de tirage additionnel qu'un emprunteur à 20%. Le total de \$24.5 Mrd d'exposition est cohérent avec le volume de prêts LendingClub sur 2007–2019.

IFRS 9 Staging : La règle SICR combinée ($PD > 3 \times PD$ origination OU $PD > Q75$) est documentée et justifiable pour un auditeur. La distribution entre les 3 stages n'est pas disponible dans le PDF, mais le seuil SICR absolu (quantile 75 de la PD 12 mois) assure mécaniquement qu'au moins

25% du portefeuille passe en Stage 2 — ce qui peut être conservateur selon le contexte réglementaire local.

ECL : Le calcul ECL lifetime par projection marginale sur 5 ans est l’approche correcte pour les Stages 2&3. L’utilisation du taux d’intérêt contractuel (`int_rate`) comme proxy de l’EIR est conforme à IFRS 9.B5.4.1, avec la nuance que l’EIR réel devrait intégrer les frais d’origination. Les résultats ECL finaux sont dépendants de la qualité du LGD — dont on a vu les limites — et sont donc à interpréter avec prudence jusqu’à correction du modèle LGD.